

We confirm and declare that this report and the project work is entirely the product of our own efforts and we have not used or presented the work of others herein without acknowledgement

EE580 Mini Project

24 February 2026

Department of Electronic and Electrical Engineering
University of Strathclyde, Glasgow

Daniel Stebbings
reg. 202118874

Preben Rasamanickam
reg. 202114642

Contents

1 Introduction	1
2 MATLAB Implementation	1
2.1 Filter Design	1
2.2 Filter Results	2
3 Embedded Implementation	2
3.1 Signal Generation	2
3.2 FIR Filter	2
3.3 FIR Filter Results	3
3.4 Convolution	3
3.5 Profiling of C implementations	3
4 Comparison	4
4.1 MATLAB versus C implementation	4
4.2 Discussion	4
5 Conclusion	4
6 Figures and Tables	6
6.1 Part 1	6
6.1.1 Input Signals	6
6.1.2 Filter Design	7
6.1.3 Filter Results	11
6.2 Part 2	12
6.2.1 Input Signals	12
6.2.2 Filter Results	14
6.2.3 Convolution	16
6.3 Part 3	18
6.3.1 Filtered Signals Time Domain	18
6.3.2 Filtered Signals Frequency Domain	19
7 Code Listings	20
7.1 MATLAB Implementation	20
7.2 C Implementation	26
7.2.1 filter.h	26
7.2.2 filter.c	29
7.2.3 util.h	34
7.2.4 util.c	37
7.2.5 main.c	40
7.3 MATLAB Comparison	44
8 Bibliography	48
9 Contributions	48

1 Introduction

The task provided centered around the implementation of Finite Impulse Response (FIR) filters for time domain signals. The signals to be used were derived from the student numbers of the students completing the task. Filters were created for each signal using MATLAB Filter Designer and tested within MATLAB. These filter coefficients were then exported as C header files and code for time domain filtering using the OMAP-L148 Digital Signal Processing (DSP) development boards was developed and optimised using Code Composer Studio (CCS). The results of this were then compared against the MATLAB implementation to find the error.

Multiple optimisation methods were investigated, including both fixed point representation and leveraging Single Instruction Multiple Data (SIMD) instructions.

2 MATLAB Implementation

2.1 Filter Design

The input signal was constructed by repeating each student's registration number one hundred times. The student numbers used are shown in Equation 1 and the resulting signal vectors are shown in Equation 2. The mean of each vector was subtracted to ensure they had no DC offset. The resulting signals are shown in Figure 1.

$$\begin{aligned} S_1 &= [2, 0, 2, 1, 1, 8, 8, 7, 4] \\ S_2 &= [2, 0, 2, 1, 1, 4, 6, 4, 2] \end{aligned} \tag{1}$$

$$\begin{aligned} x_1 &= [2, 0, 2, 1, 1, 8, 8, 7, 4, 2, 0, 2, \dots, 8, 7, 4] \\ x_2 &= [2, 0, 2, 1, 1, 4, 6, 4, 2, 2, 0, 2, \dots, 4, 6, 4] \end{aligned} \tag{2}$$

The sampling rate was calculated by rounding the average of the final four digits of each student number as shown in Equation 3. The resulting time domain plots are shown in Figure 1. An 1024 point Fast Fourier Transform (FFT) of each signal was computed, as shown in Figure 2. It was found that both signals had a peak at 1504 Hz, and a second peak at 752 Hz. In accordance to the provided brief, the 1504 Hz peak in x_1 was selected for suppression, and the 752 Hz peak in x_2 . A zoomed plot of the relevant peaks is shown in Figure 3.

$$f_s = \left\lfloor \frac{S_1[5 : 8] + S_2[5 : 8]}{2} \right\rfloor = 6758 \tag{3}$$

For each signal an FIR bandstop filter was developed using MATLAB Filter Designer with a stop band limited to 100Hz. For x_1 , the peak had a skew towards higher frequencies, so the stop band was defined from -50 Hz to +20 Hz of the 752 Hz. The implemented filter, h_1 , had an attenuation of over 70 dB in the stop band with an order of 358.

For x_2 , the peak was slightly skewed as well, and the stop band defined from -40 Hz to +20 Hz of the 1504 Hz peak. The implemented filter h_2 had an attenuation of over 60dB in the stop band and an order of 314, although it had quite a high ripple in the stop band.

Although the filters had high coefficient counts, there was no real time requirement imposed by this project. Further, this provided a good benchmark to optimise against.

The filter coefficients / impulse response of the filters are shown in Figure 4. Both filters were symmetric with a large central coefficient and much smaller ripples to either side. The magnitude and phase response of h_1 and h_2 are shown in Figure 6 and Figure 9 respectively. Both filters had minimal pass band ripple. They both also have linear phase, although with a nonlinearity in the stop band. As these were both FIR filters the pole zero diagrams, as shown in Figure 5 and Figure 7 / Figure 8, had all poles at the root and several zeros in complex conjugate pairs around the unit circle. The zeroes associate with the stop band are arrayed at a correspondingly lower frequency for h_1 than h_2 .

2.2 Filter Results

The signals were filtered using Equation 4 to create y_1 and y_2 .

$$y_k = h_k * x_k \quad (4)$$

3 Embedded Implementation

3.1 Signal Generation

The input signals x_1 and x_2 were constructed using a methodology similar to Section 2.1, and stored in arrays for subsequent processing.

The generation process involved calculating the mean of the student's registration number shown in Equation 1. Each digit of the registration number was then offset by the mean and assigned to the signal array. This sequence was then copied across the signal array until the target signal length was reached.

In CCS, time domain plots were generated for three cycles of the signals, see Figure 13 and Figure 14, and frequency domain plots of the full signals, see Figure 15 and Figure 16.

3.2 FIR Filter

The filter coefficients were derived as described in Section 2.1 and stored in `data_1_fixed.h` and `data_2_fixed.h`. The filter specifications were encapsulated in a structure definition containing the number of coefficients, a pointer to the coefficient array, and its symmetric properties.

The FIR filter implementation employs a tapped delay line architecture. In this architecture, each input sample is multiplied by its corresponding filter coefficient and added to an accumulator. Once all taps are processed, the accumulator value is stored in the output array as the filtered sample. After all input samples have been passed through the delay line, the output array contains the filtered signal.

The tapped delay line implementation uses a nested for-loop:

- **Outer-loop:** Iterates through the length of the input signal.
- **Inner-loop:** Iterates through the delay line samples, and performs a Multiply-Accumulate (MAC) operation to process the tap and accumulator.

To improve efficiency, the inner loop was optimised to only iterate through the samples currently present in the delay line. This avoids redundant MAC operations with the initial zero values.

Further optimisations was achieved by implementing the following measures:

- **Symmetry:** By “folding” the tapped delay line for symmetric filter coefficients, the number of multiplications was halved.
- **Fixed-Point Integer Arithmetic:** Converting the signals and filter coefficients to integers accelerates arithmetic operations by reducing the required CPU cycles. This measure increases the error of the output.
- **Loop Unrolling & Intrinsic:** Utilising compiler intrinsics for parallelisation, pipelining, and reducing memory accesses. This reduces the required CPU cycles to run the filter.

The implementations can be seen in Section 7.2.

3.3 FIR Filter Results

The filtered signal time domain plots can be seen in Figure 13 and Figure 14, and frequency domain plots can be seen in Figure 15 and Figure 16.

The following observations confirms the correctness of the implementation:

- The time domain plots in MATLAB and CCS are visually similar, see Figure 1 and Figure 13 respectively.
- The frequency domain plots confirm that the targeted frequencies have been attenuated, matching the intended filter response, see Figure 15 and Figure 16.

3.4 Convolution

While the standard FIR filter implementation stops after the last input sample is passed into the tapped delay line, a full convolution requires the entire tapped delay line to be flushed before the result is produced.

To demonstrate the convolution, two 4 sample integers were convolved. The input signals contain the values in Equation 5, and the resulting convolution is given in Equation 6.

$$\begin{aligned} x_1 &= [1, 1, 1, 1] \\ x_2 &= [1, 2, 1, 2] \end{aligned} \tag{5}$$

$$\text{Convolution} = [1, 3, 4, 6, 5, 3, 2] \tag{6}$$

The step-by-step process with a visual of the DSP memory state can be seen in Figure 21 and the following figures.

3.5 Profiling of C implementations

The various C implementations mentioned in Section 3.2 were profiled using the number of clock cycles and Mean Squared Error (MSE) compared to MATLAB. The standard FIR filter implementation was used as a baseline for evaluating improvements in speed and trade-offs in accuracy. The results can be seen in Table 1 below.

Table 1: Performance comparison of FIR filter C implementations

Implementation	Number of CPU cycles	Comparison (%)	MSE
Standard	15315458	100.0	1.254e-07
Symmetric	15393043	100.5	1.253e-07
Symmetric Fixed point	14427900	94.20	1.217e-02
SIMD Fixed point	7219817	47.10	1.218e-02

From the results, it can be observed that converting to fixed point reduces signal integrity significantly. However, by applying SIMD intrinsics a large efficiency increase in run time of 50% can be achieved.

4 Comparison

4.1 MATLAB versus C implementation

The C filtered signals were recorded to a .csv file and loaded into MATLAB for error analysis.

The time domain and error per sample of the C signals can be seen in Figure 25 and Figure 26. It can be observed that there is a small fluctuating deviation between the signals.

The frequency domain and frequency response error of the C signals can be seen in Figure 27 and Figure 28. It can be observed that there is a small deviation between the signals near the frequency peaks of the original signal.

To quantify the time domain deviation between MATLAB and the C implementation, the MSE of the filtered signals was calculated over the full length. The results are given in Equation 7 below.

$$\begin{aligned} \text{MSE}_1 &= 1.254e - 07 \\ \text{MSE}_2 &= 5.654e - 08 \end{aligned} \tag{7}$$

4.2 Discussion

The primary driver of the observed time and frequency domain error is the difference in numerical precision, see Figure 26 and Figure 28. MATLAB defaults to 64-bit double-precision floating-point, whereas the C67x DSP hardware uses 32-bit single-precision floating-point. This results in the arithmetic operations in the tapped delay line introducing a small quantisation error that accumulates over the 300+ taps of the filter. However, for most practical applications, an MSE in the magnitude of 10^{-7} is considered negligible.

5 Conclusion

This project successfully demonstrated the transition of a digital filter design from MATLAB to an embedded hardware platform (OMAP-L138) optimised for DSP. The implementation confirmed that:

- FIR filters can be implemented in C for effective attenuation of targeted frequencies.
- Numerical precision trade-offs are inherent in embedded DSP. However, single-precision floating-point arithmetic provides sufficient accuracy for most applications.

- Hardware-specific optimisations, particularly SIMD, can reduce run time by over 50%, but with significant loss of signal integrity.

6 Figures and Tables

6.1 Part 1

6.1.1 Input Signals

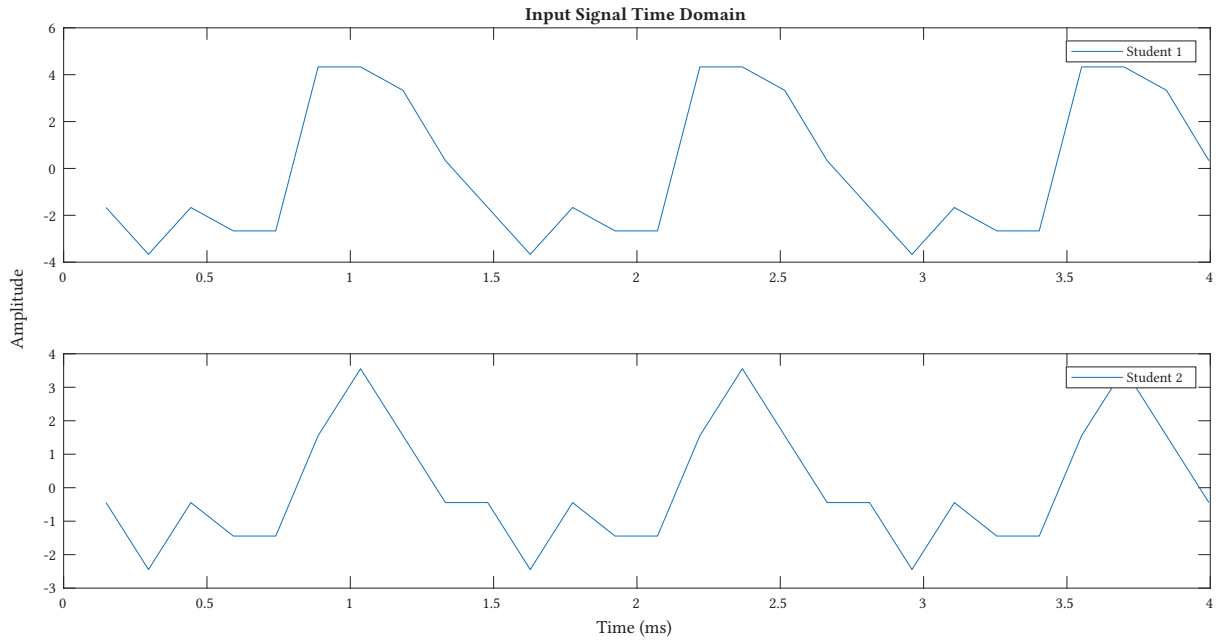


Figure 1: Input signals in time domain

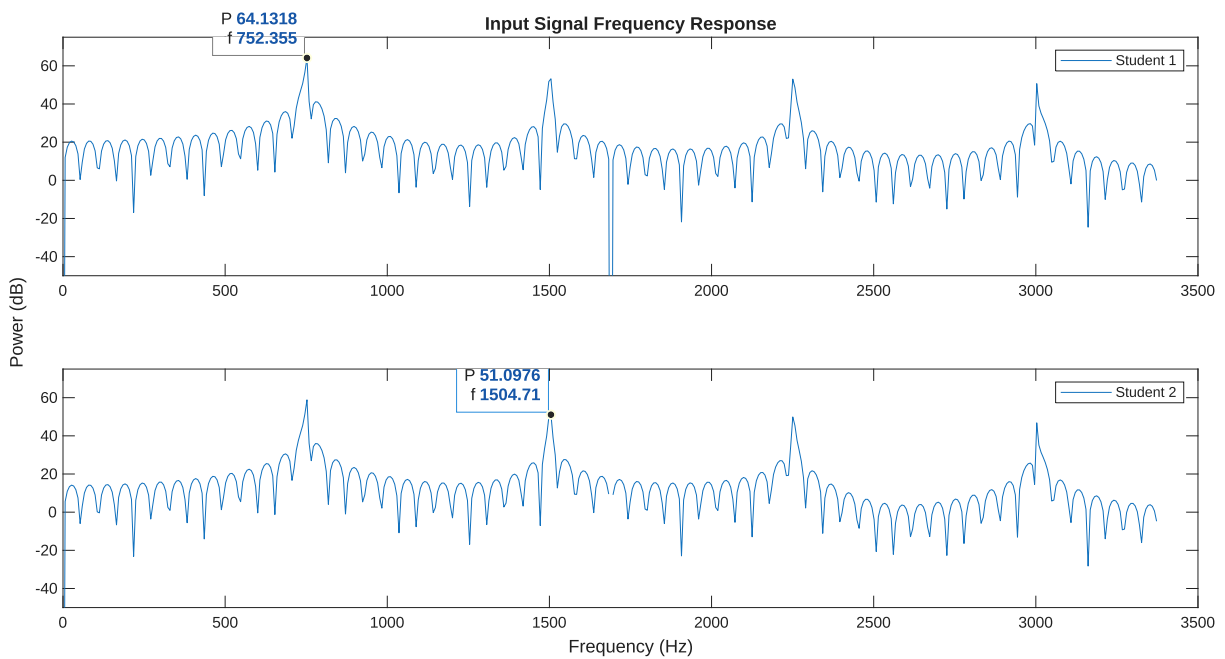


Figure 2: Input signals in frequency domain. The first peak of both signals is at 752 Hz, and the second peak of the second signal is at 1504 Hz.

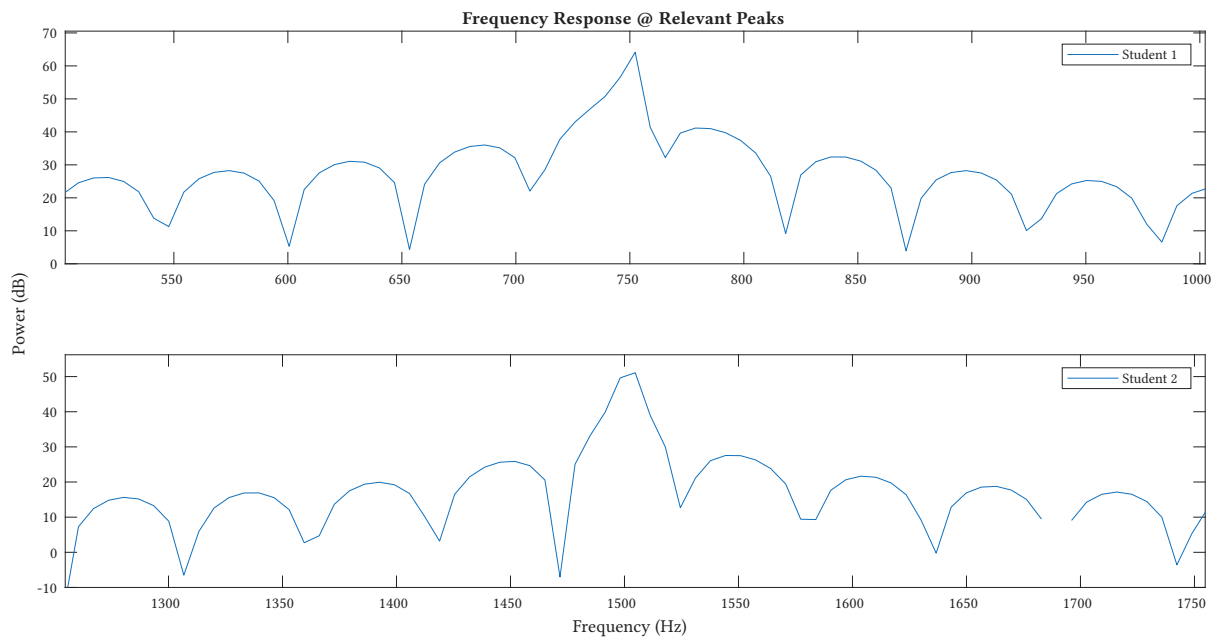


Figure 3: Frequency domain plots of input signals zoomed to relevant peaks

6.1.2 Filter Design

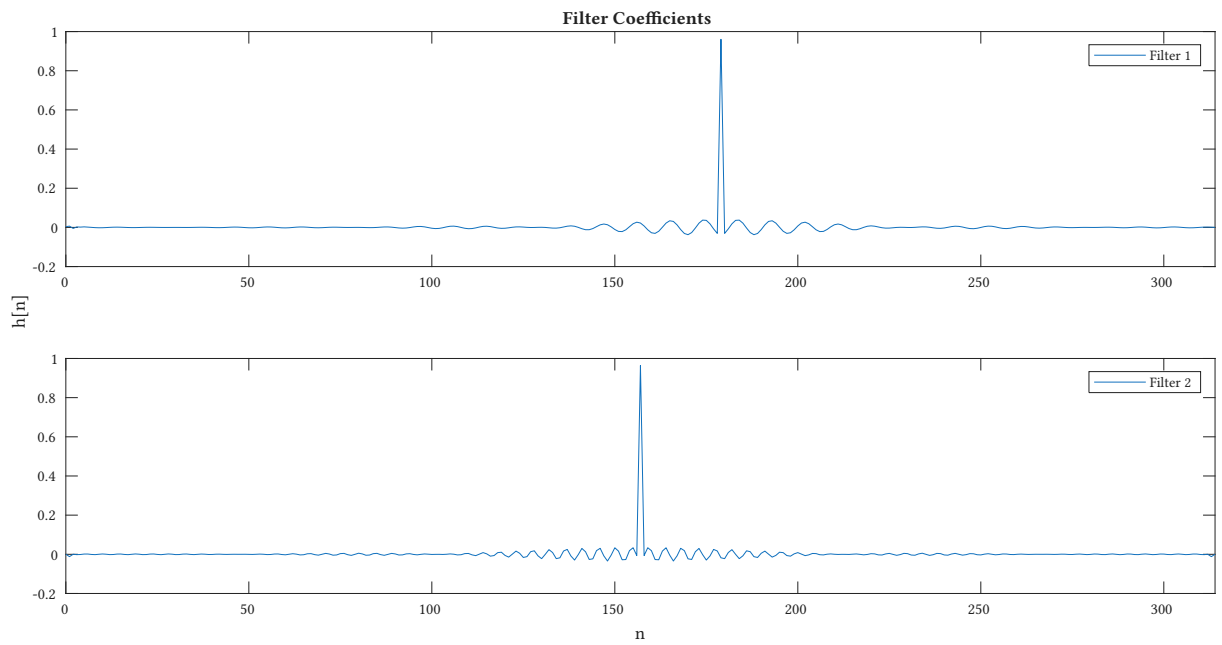


Figure 4: Coefficients of each filter

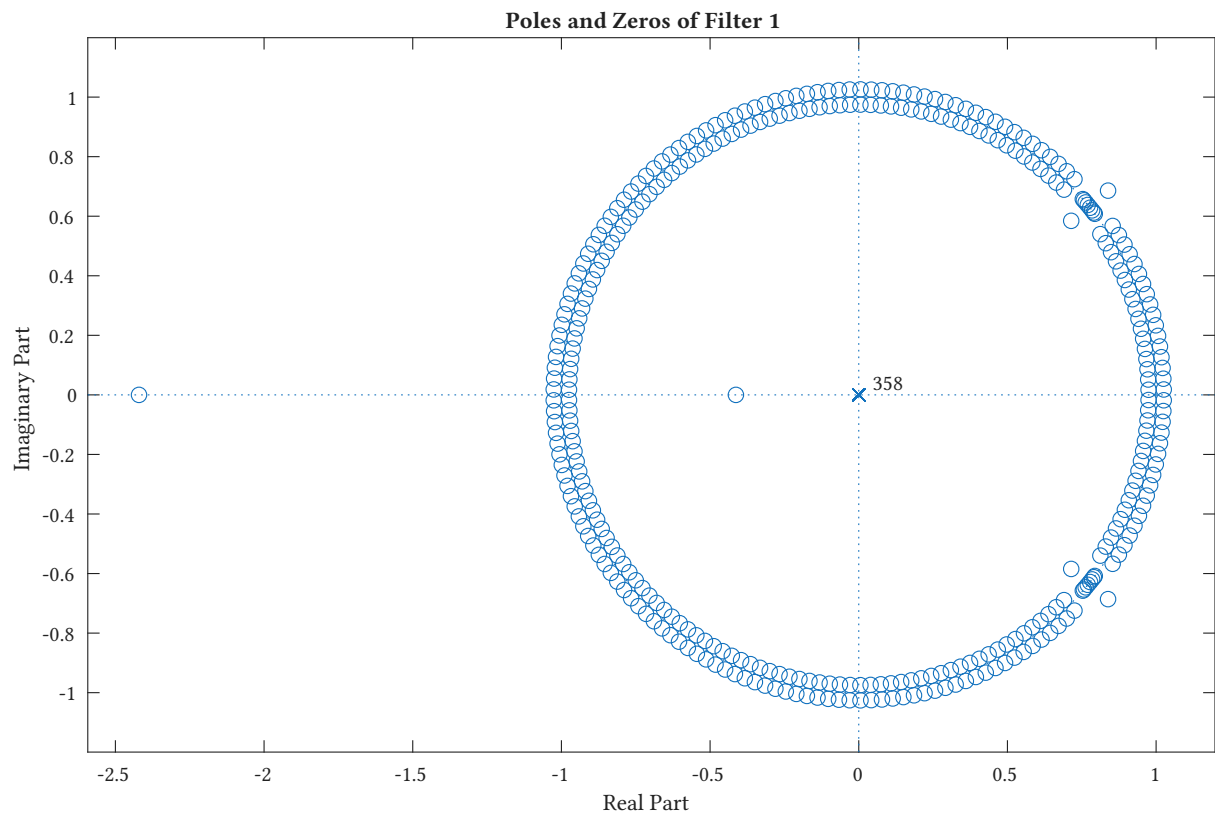


Figure 5: Poles and Zeros of H_1

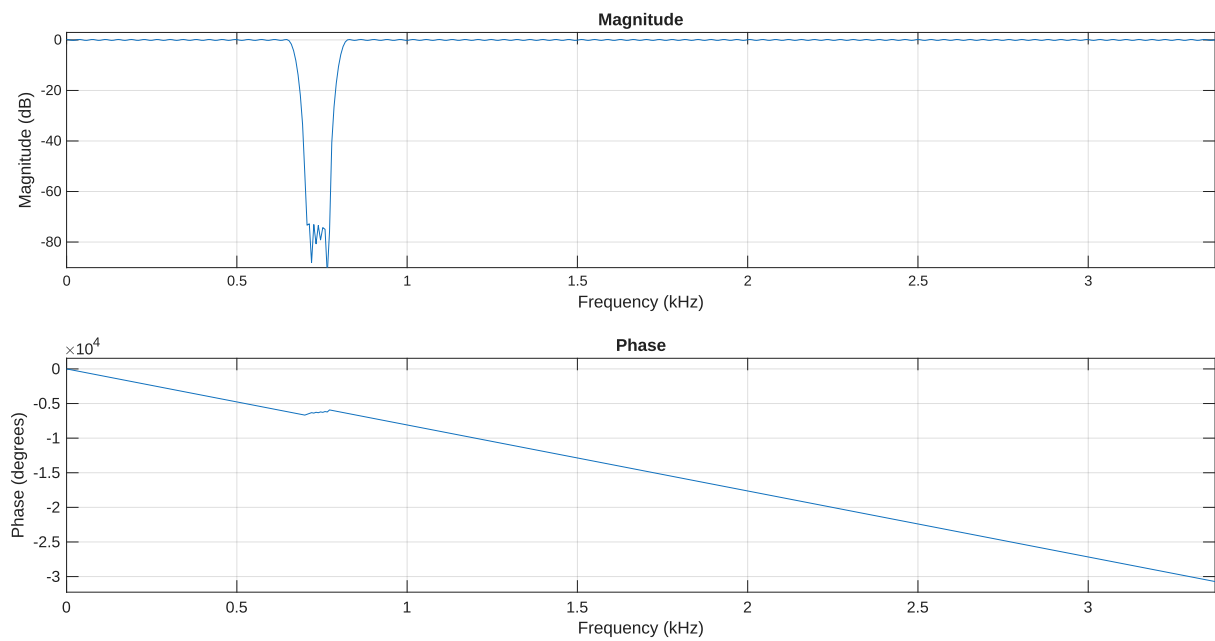


Figure 6: Magnitude and Phase response of H_1

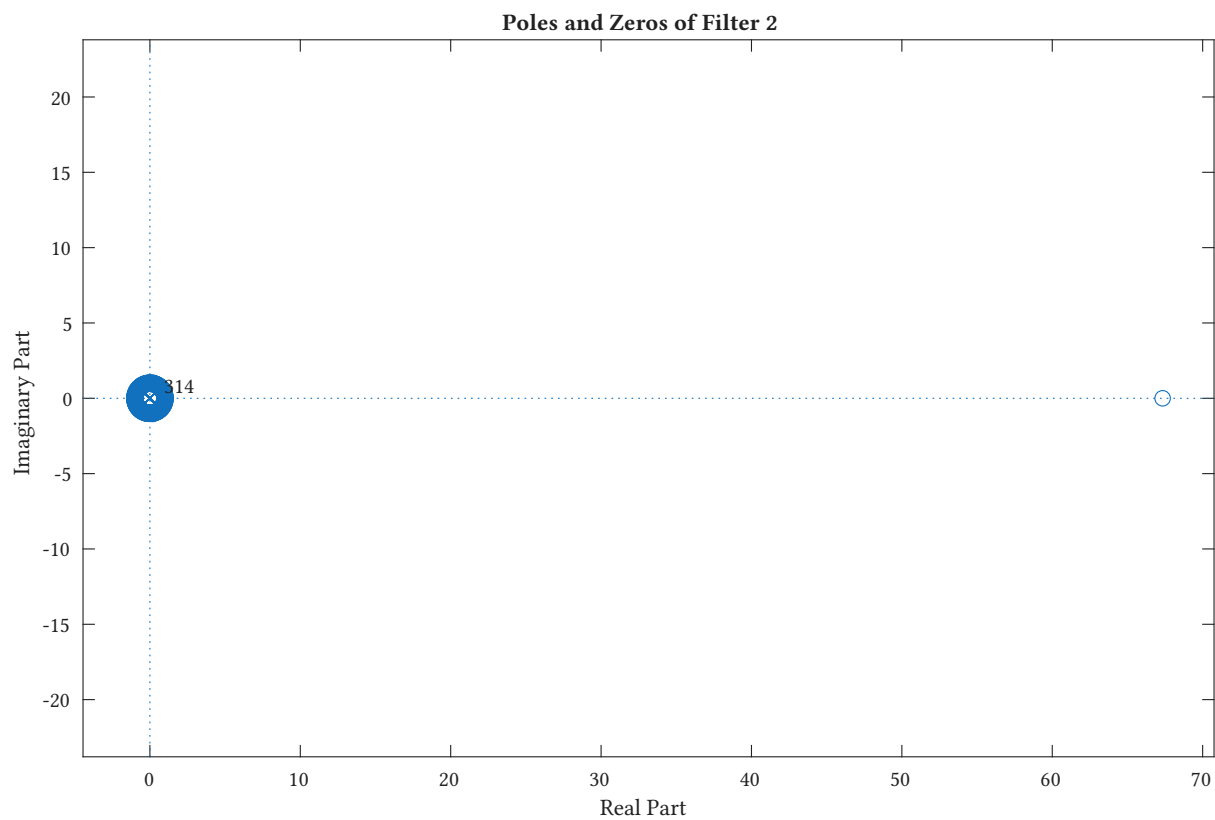


Figure 7: Poles and Zeros of H_2

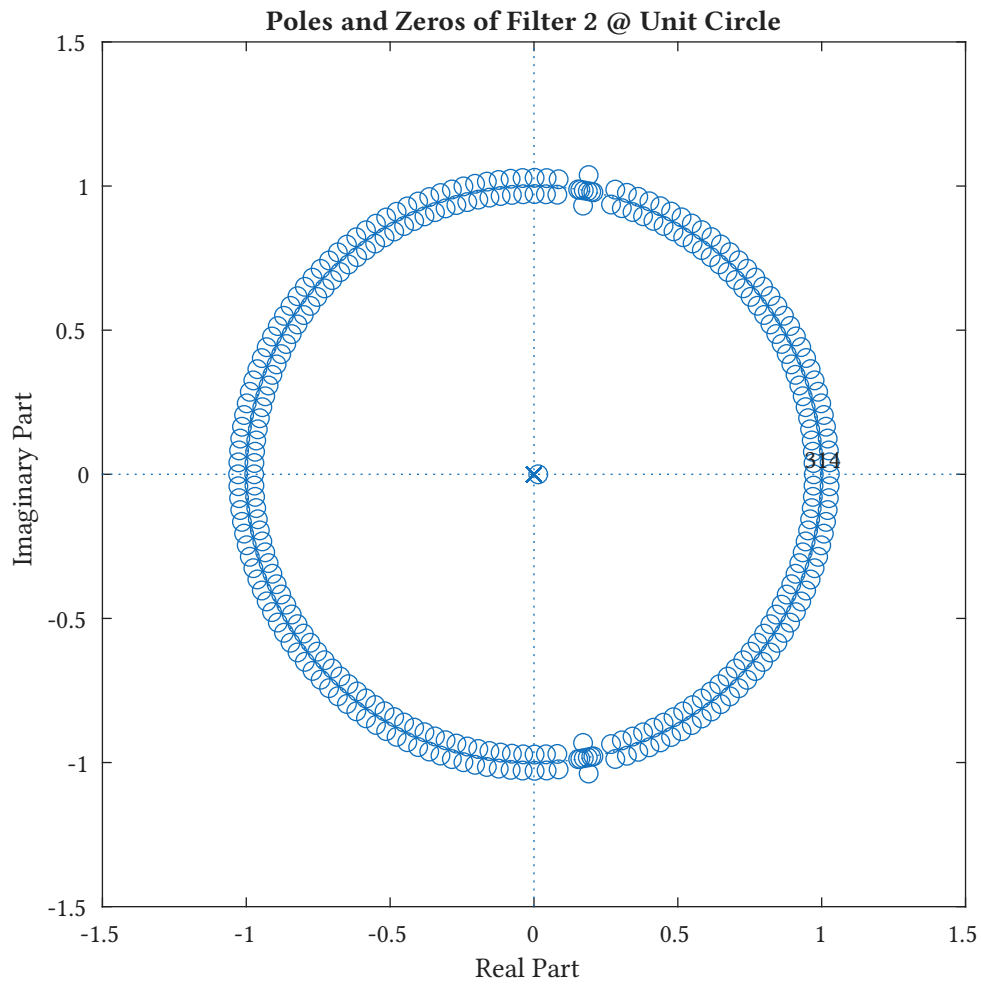


Figure 8: Zoomed Poles and Zeros of H_2

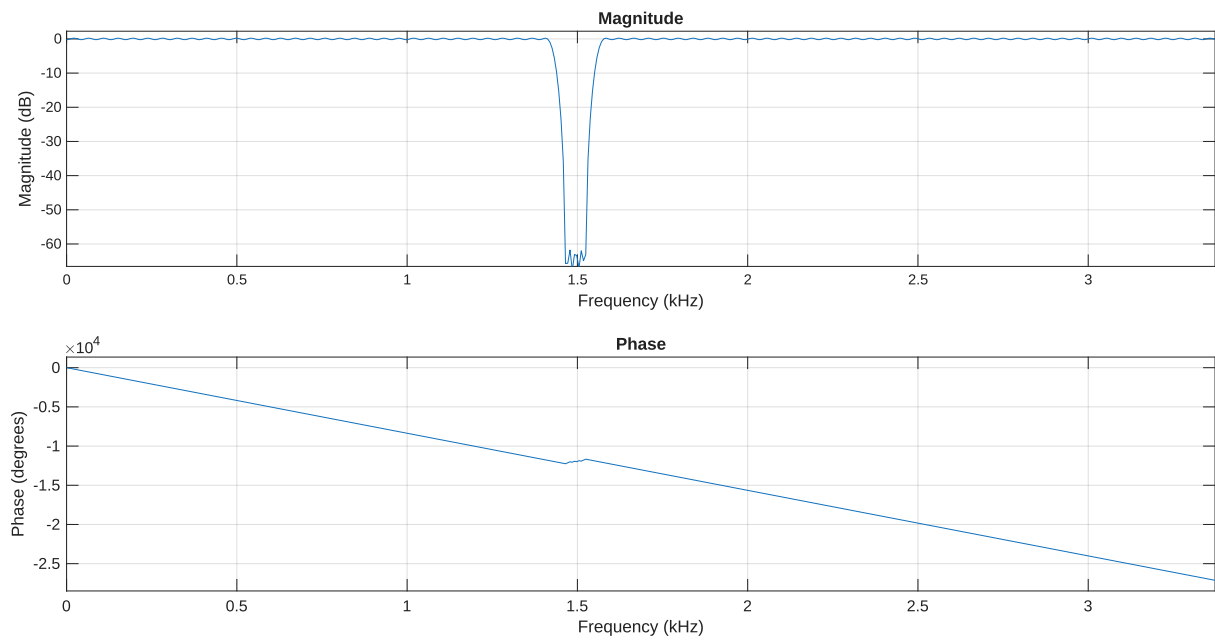


Figure 9: Magnitude and Phase response of H_2

6.1.3 Filter Results

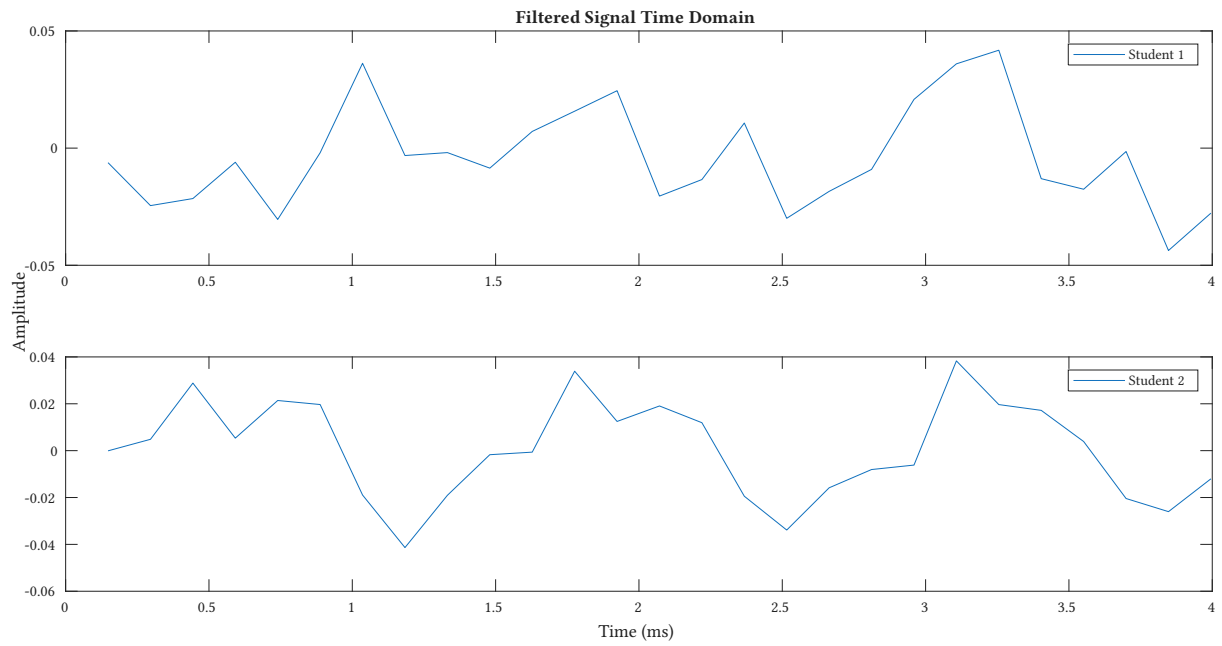


Figure 10: Time domain representation of y_1 and y_2

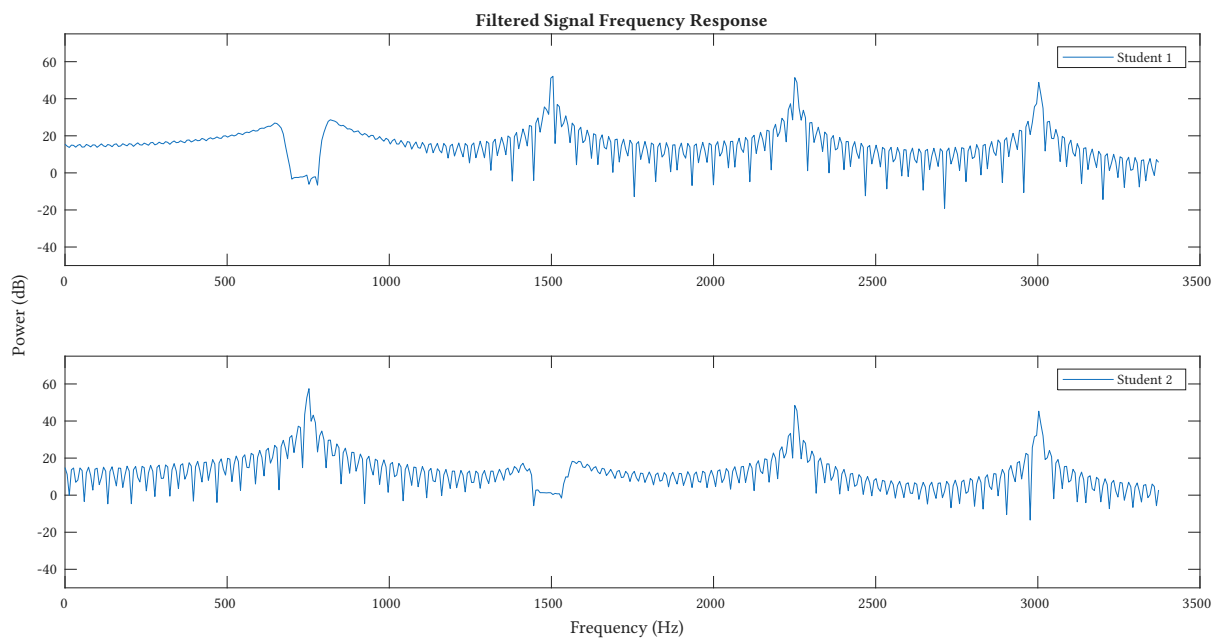


Figure 11: Frequency representations, Y_1 and Y_2

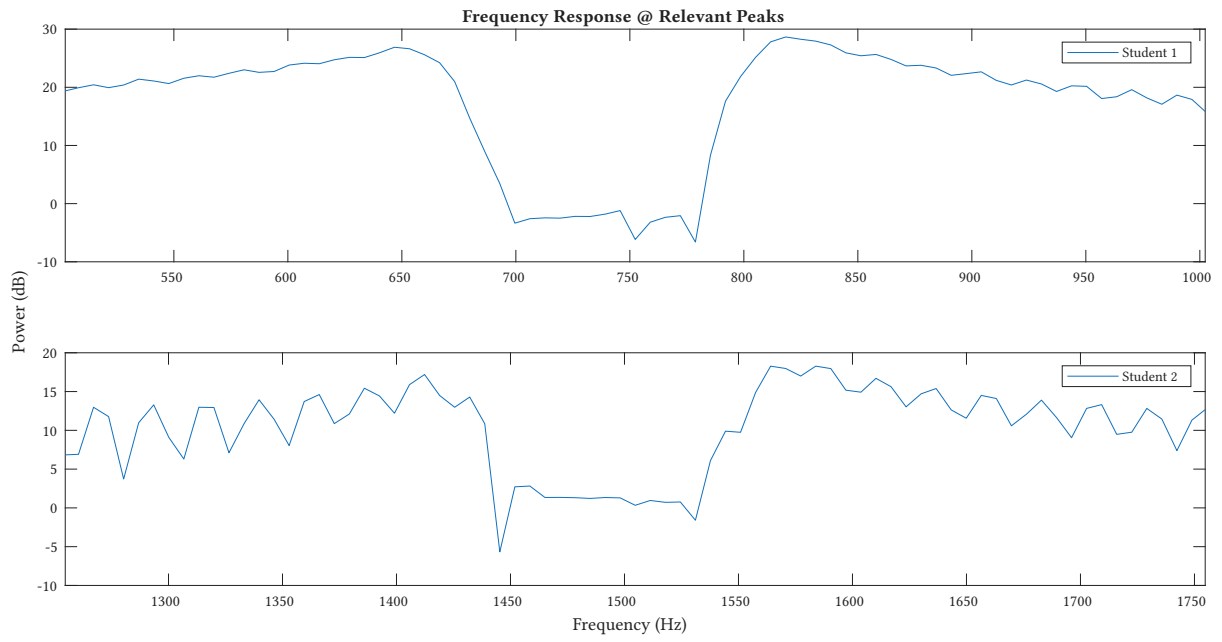


Figure 12: Zoomed frequency representations, Y_1 and Y_2

6.2 Part 2

6.2.1 Input Signals

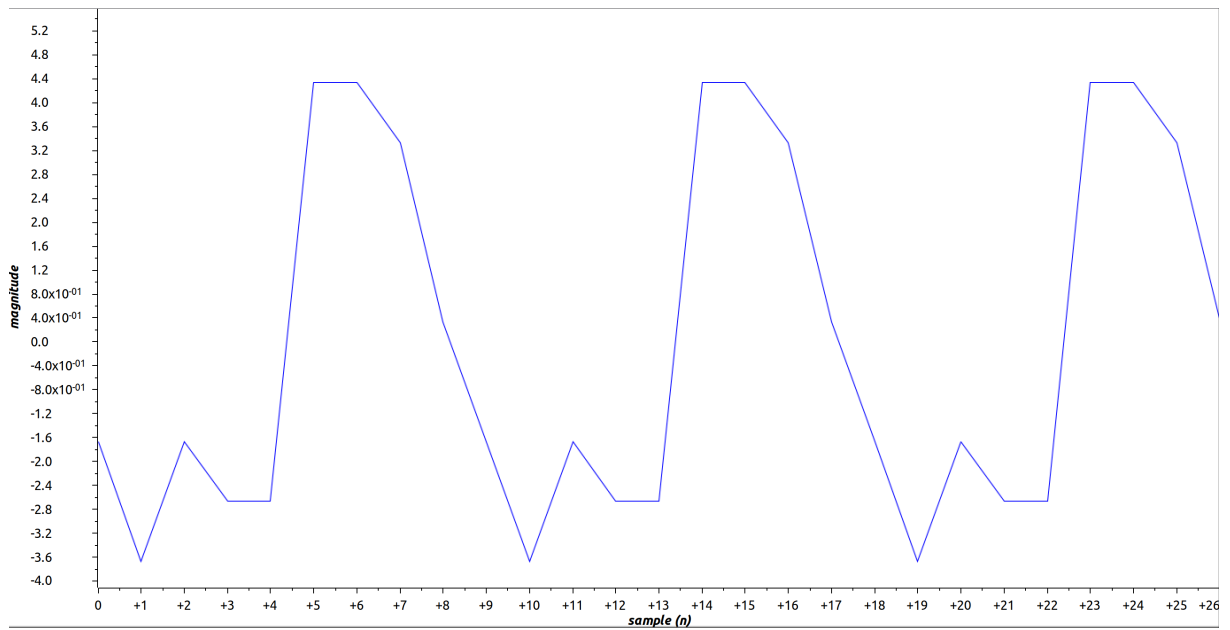


Figure 13: Student 1 input signal in time domain from CCS

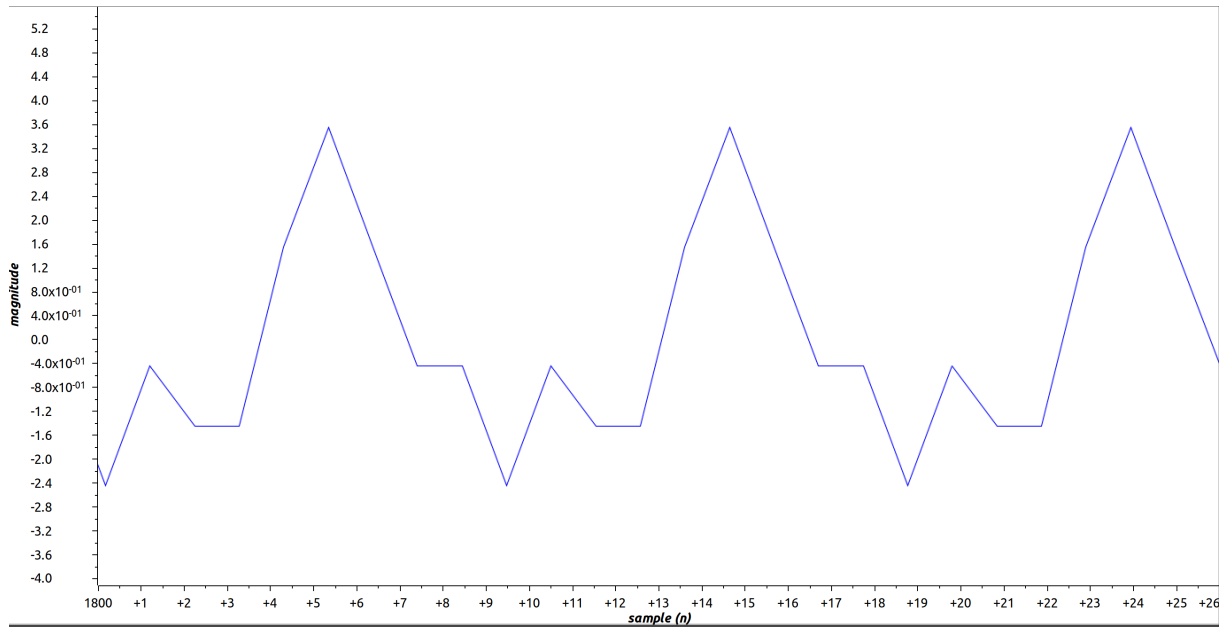


Figure 14: Student 2 input signal in time domain from CCS

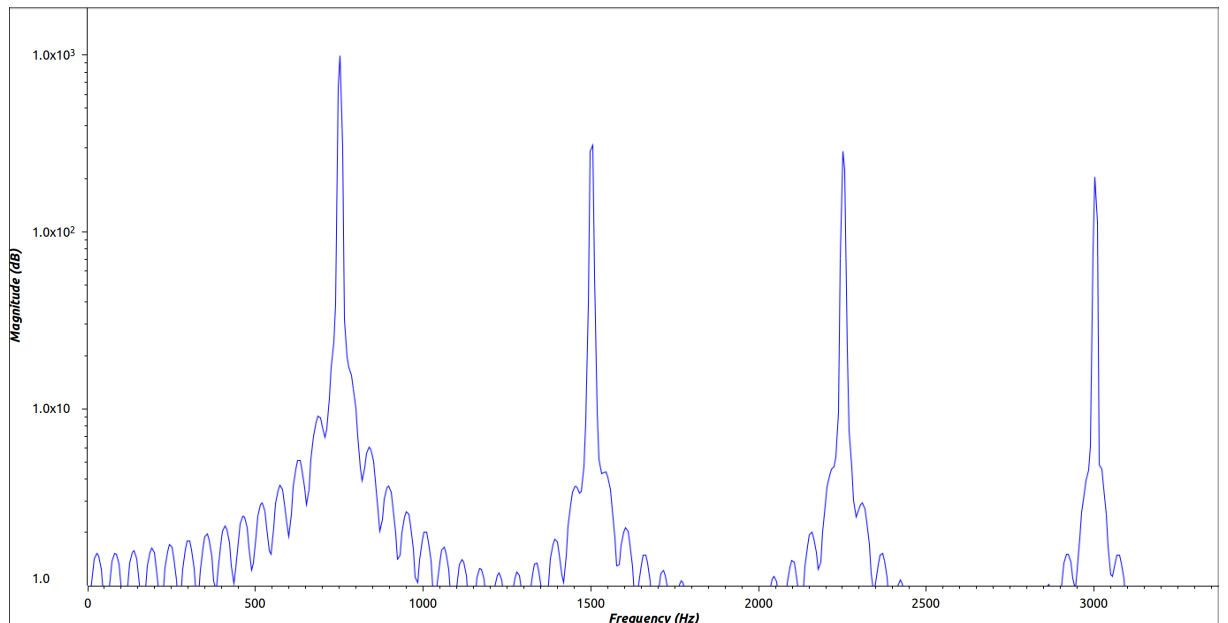


Figure 15: Student 1 input signal in frequency domain from CCS

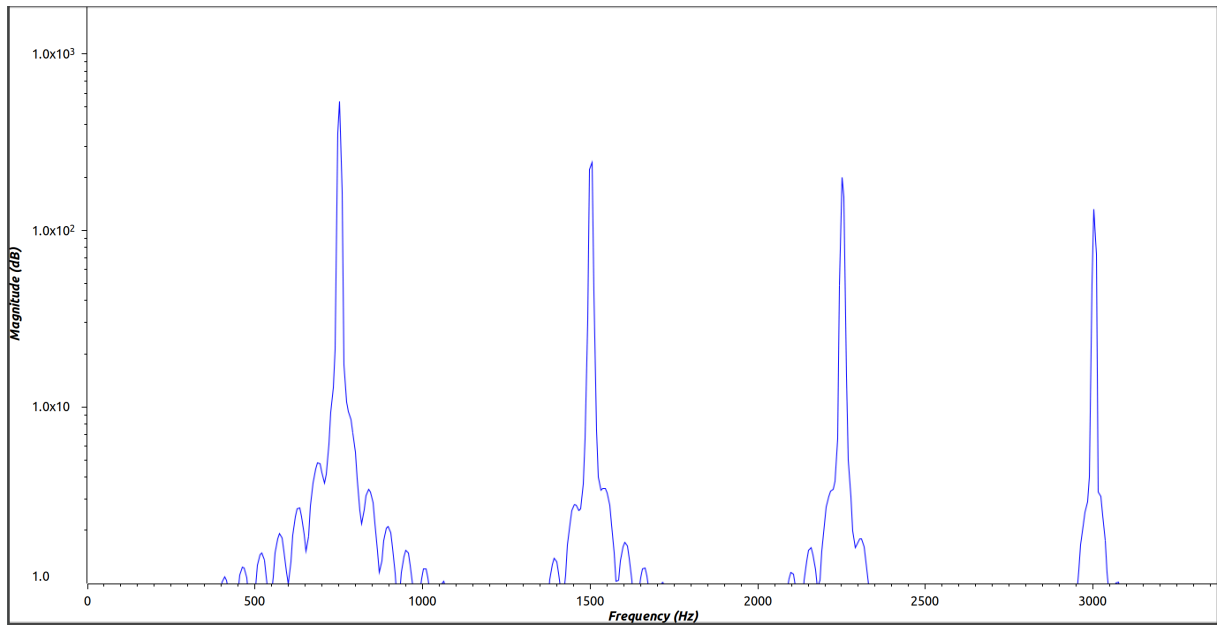


Figure 16: Student 2 input signal in frequency domain from CCS

6.2.2 Filter Results

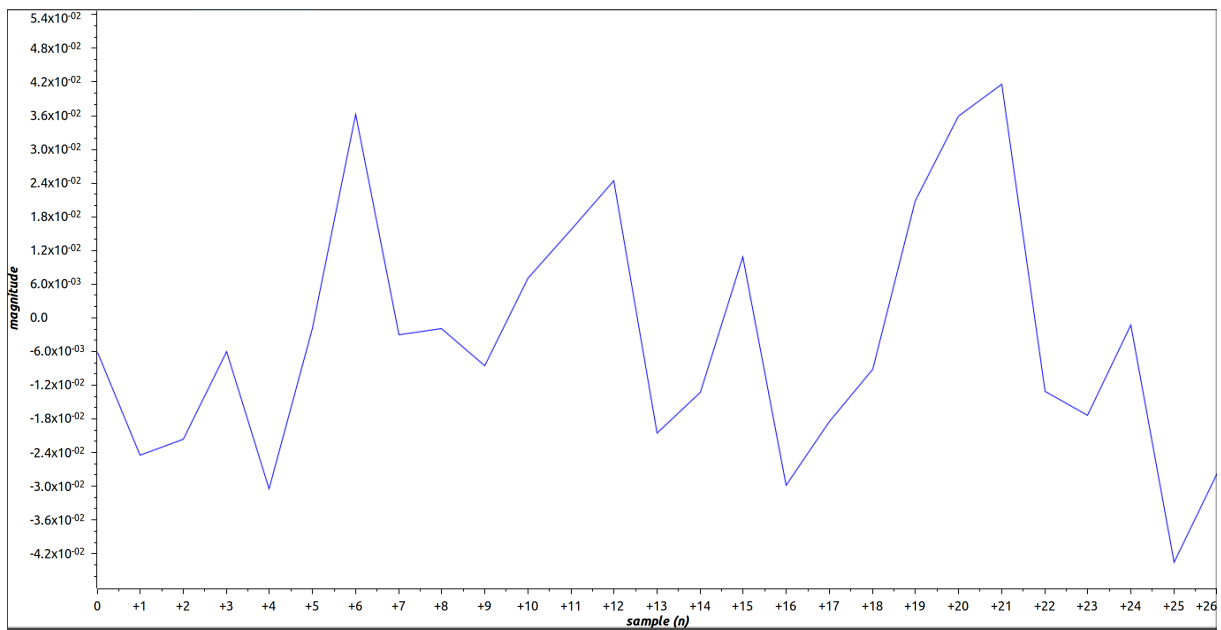
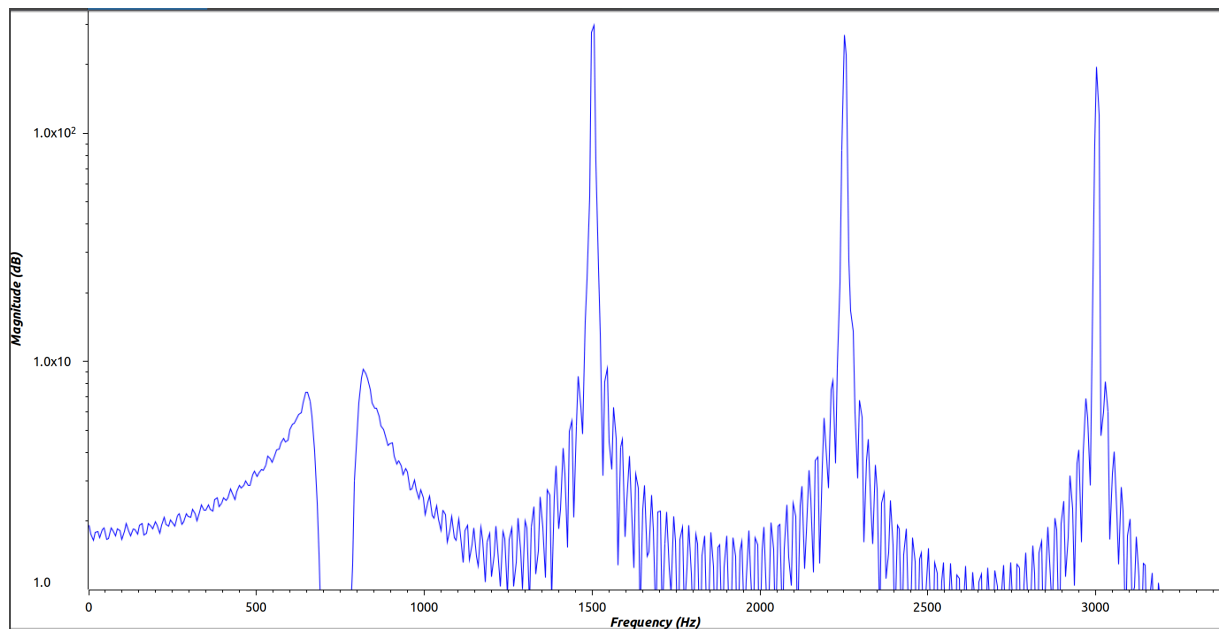
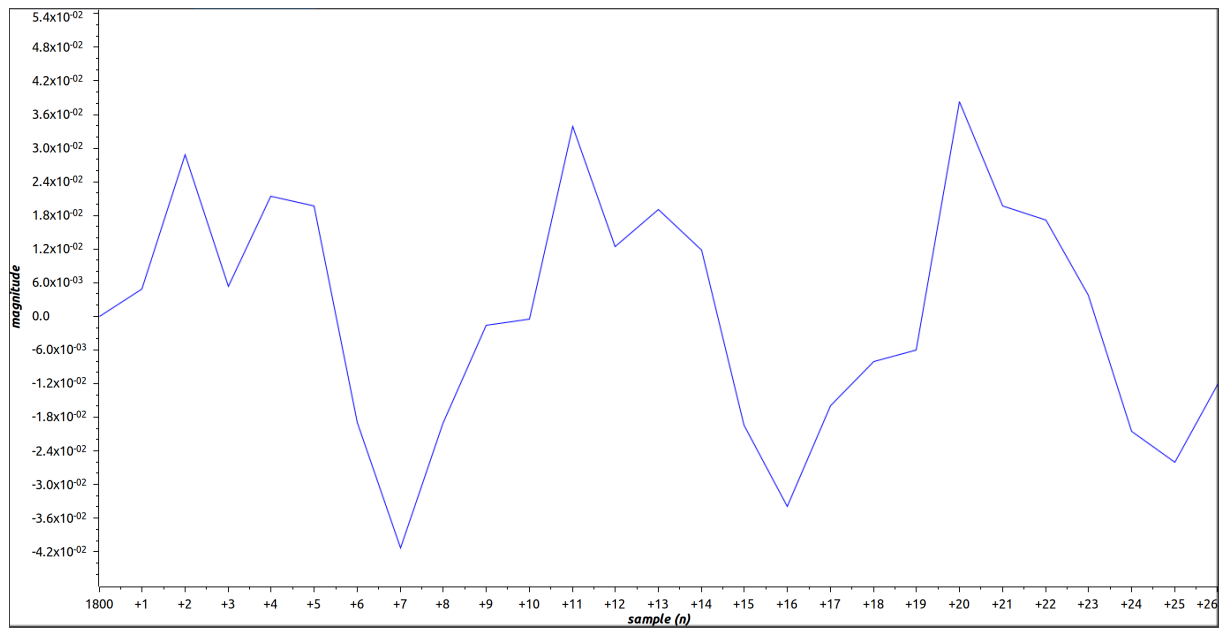


Figure 17: Student 1 output signal in time domain from CCS



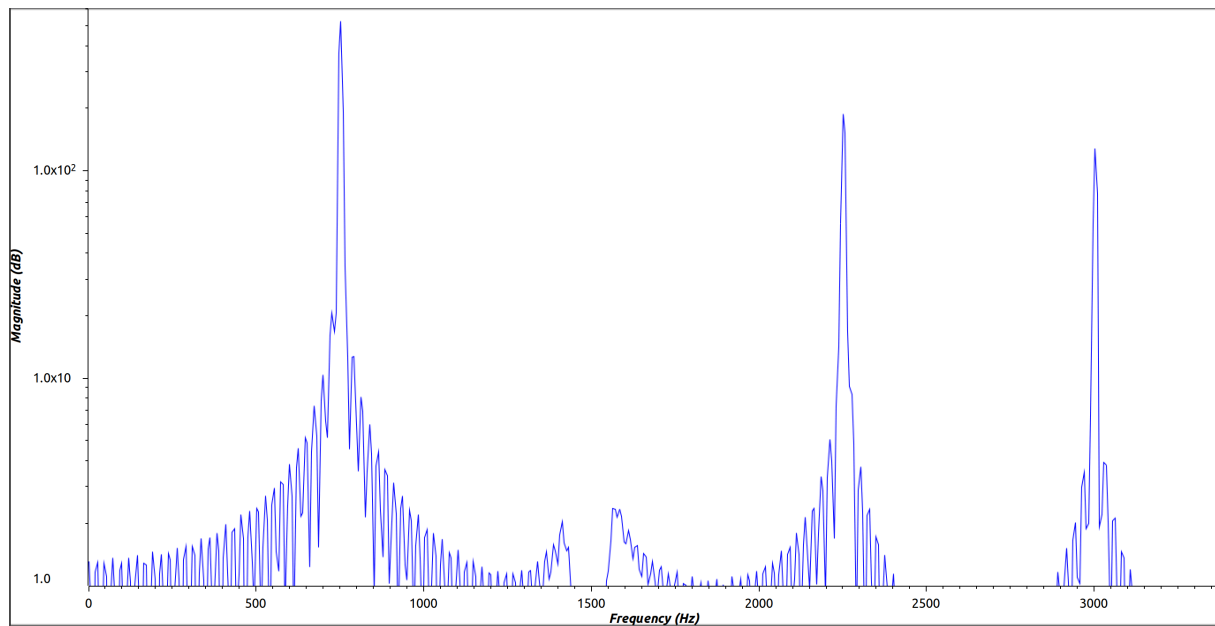


Figure 20: Student 2 output signal in frequency domain from CCS

6.2.3 Convolution

```

21     }
22     else break;
23 }
24
25     p_output[n] = l_acc;
26 }
27 }
28
29
30 void convolution(const int32_t *p_input1, uint32_t p_input1_len, const int32_t *
31 {
32     *p_output_len = p_input1_len + p_input2_len - 1;
33
34     uint32_t n, k;
35     for (n = 0; n < *p_output_len; n++)
36     {
37         uint32_t l_acc = 0;
38
39         for (k = 0; k < p_input1_len; k++)
40         {
41             if (n >= k && (n - k) < p_input2_len)
42             {
43                 l_acc += p_input1[k] * p_input2[n - k];
44             }
45         }
46         p_output[n] = l_acc;
47     }
48 }
49
50 void symm_filter_signal(const float32_t *p_input, uint32_t p_input_len, const fil
51 {

```

Expression	Type	Value	Address
n	unsigned int	0	0x0080EFC4
k	unsigned int	0	0x0080EFC8
p_input1[k]	int	1	0x0080F008
p_input1_len	unsigned int	4	0x0080EFB0
p_input2[n-k]	int	1	0x0080F018
p_input2_len	unsigned int	4	0x0080EFB8
p_output[n]	int	0	0x0080F028
p_output_len	unsigned int*	0x0080F048 {7}	0x0080EFC0
l_acc	unsigned int	0	0x0080EFC4

Figure 21: Convolution - start state in CCS

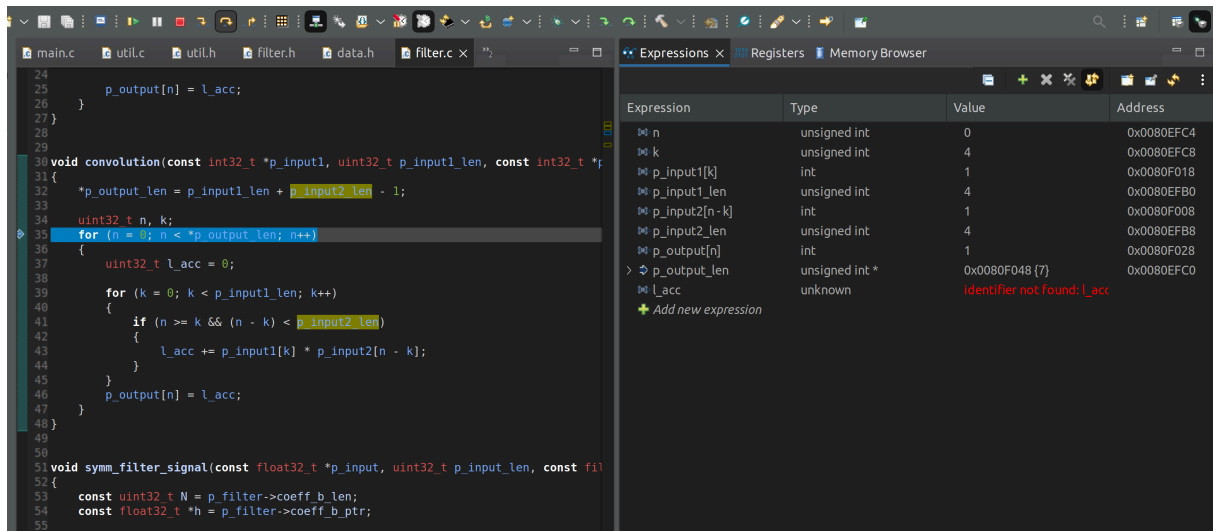


Figure 22: Convolution - first output sample in CCS

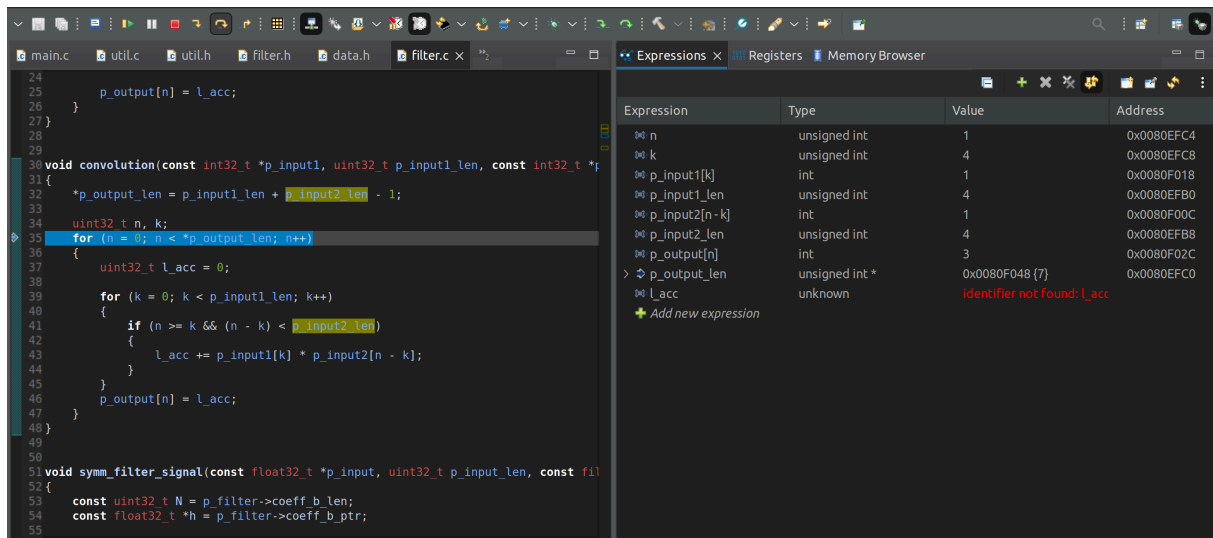


Figure 23: Convolution - second output sample in CCS

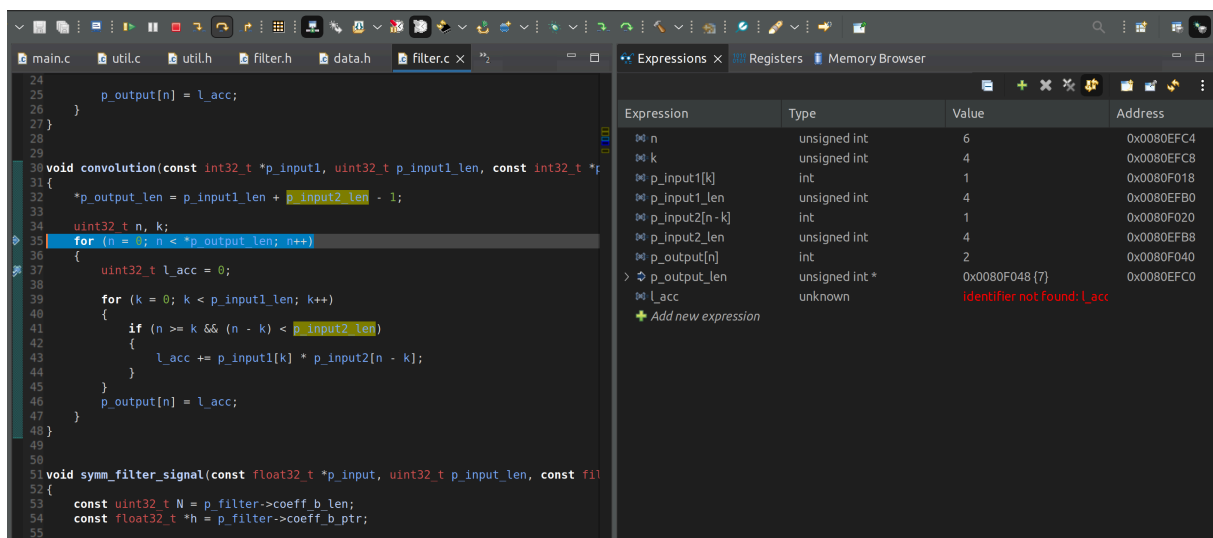
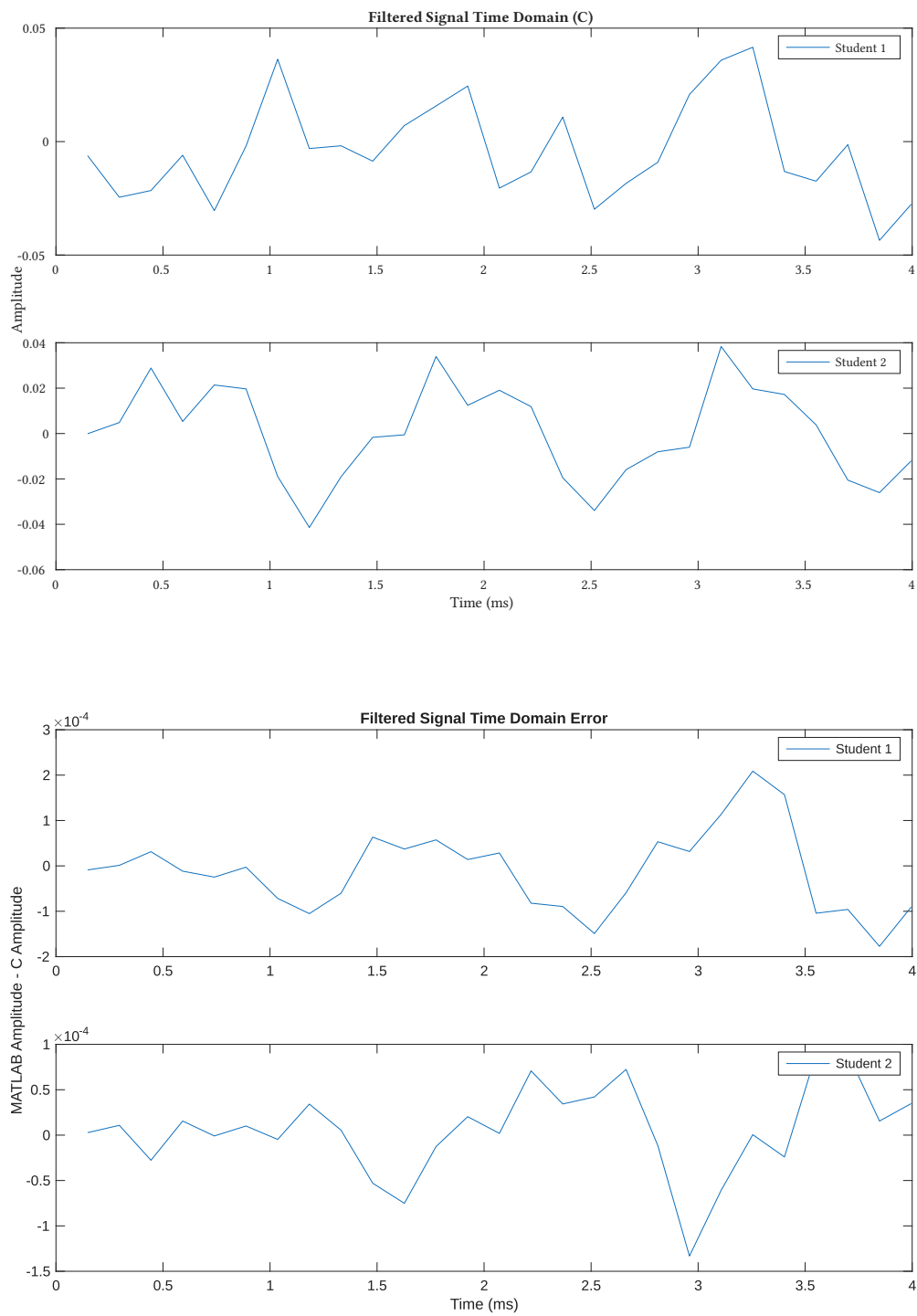


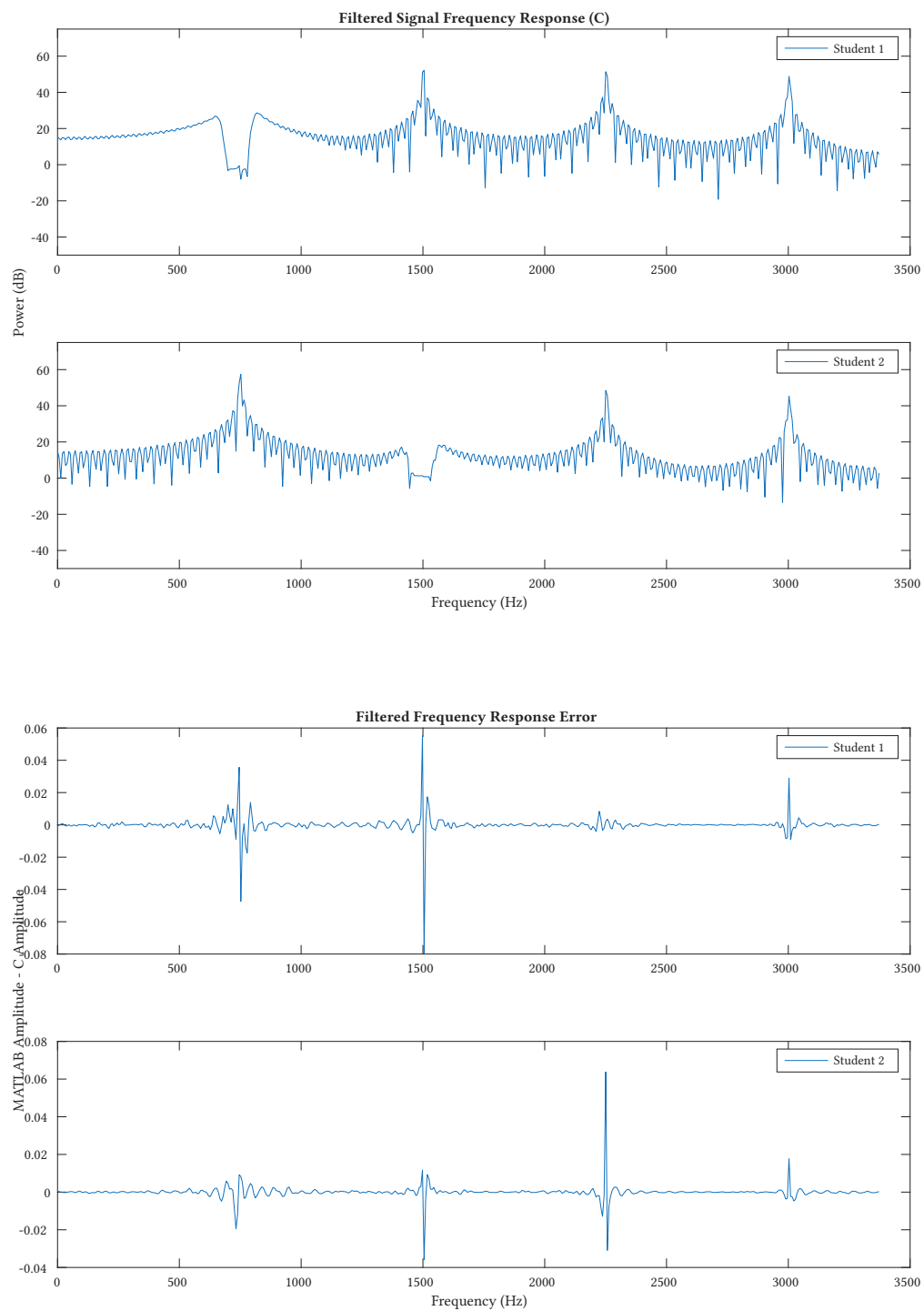
Figure 24: Convolution - last output sample in CCS

6.3 Part 3

6.3.1 Filtered Signals Time Domain



6.3.2 Filtered Signals Frequency Domain



7 Code Listings

7.1 MATLAB Implementation

```
%[text] # EE580 Data Generation & Filtering
%[text] ## Input Signals
%[text] "In Matlab, generate two signals, x1\[n\] and x2\[n\], of 900 float
elements each. The samples in a signal are the 9 digits in your student
number, repeated 100 times. For example, if the first student number is
"201338959", the first signal is x1\[n\]=\[2, 0, 1, 3, 3, 8, 9, 5, 9, 2, 0, 1,
3, 3, 8, 9, 5, 9, 2, 0, 1, 3, 3, 8, 9, 5, 9, ...]. Signal x2\[n\] will be
generated using the second student number. From each signal, subtract its mean
value, so that the resulting signal has a mean equal to zero. "
%%
reg1 = [2 0 2 1 1 8 8 7 4];
reg2 = [2 0 2 1 1 4 6 4 2]; % Placeholder

x1 = repmat(reg1,1,100);
x2 = repmat(reg2,1,100);

% Subtracting mean so zero centered
x1 = x1 - mean(x1);
x2 = x2 - mean(x2);
%[text] ## Sampling Frequency
%[text] The sampling frequency to be used is equal to the rounded mean between
the values identified by the last four digits in both student numbers. For
example, if the first student number is "201338959" and the second student
number is "201343242", the sampling frequency to be used is
round((8959+3242)/2)=6101Hz. In those cases where the sampling frequency
results in being less than 1000Hz, then 1000Hz should be added to it.
reg1_last4 = str2double(sprintf("%d",reg1(end-3:end)));
reg2_last4 = str2double(sprintf("%d",reg2(end-3:end)));
fs = round( ( reg1_last4 + reg2_last4 )/2 )

if (fs < 1000)
    fs = fs + 1000;
end
%[text] ## Plot Signals
%[text] "In Matlab, plot time graphs of 3 cycles (3 x 9 samples = 27 samples)
of each of the two signals generated; also plot frequency graphs of the two
signals (using all 900 samples available for each), using 1024 points for the
FFT. All plots should be fully and correctly labelled. "
t = (1:27)/fs;

% https://uk.mathworks.com/matlabcentral/answers/499823-one-common-xlabel-and-ylabel-for-multiple-subplots
fig = figure;
subplot(2,1,1)
plot(t.*1e3,x1(1:length(t)), "DisplayName", "Student 1" );
legend
```

```

subplot(2,1,2)
plot(t.*1e3,x2(1:length(t)), "DisplayName", "Student 2" );
legend

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Amplitude');
xlabel(han,'Time (ms)');
title(han,'Input Signal Time Domain');

X1 = fft(x1,1024);
X1_power = 10*log10(abs(X1(1:1024/2)).^2);

X2 = fft(x2,1024);
X2_power = 10*log10(abs(X2(1:1024/2)).^2);

F = (fs/1024)*(0:1024/2-1);

fig = figure;

subplot(2,1,1)
X1_plt = plot(F,10*log10(abs(X1(1:1024/2)).^2), "DisplayName", "Student 1" );
legend
ylim([-50,75])
[X1_pkval,X1_pkidx] = max(X1_power,[],"all");
X1_pkfreq = F(X1_pkidx)
X1_pktip = datatip(X1_plt,'DataIndex',X1_pkidx,'Location','northwest');
X1_plt.DataTipTemplate.DataTipRows(1) = dataTipTextRow("P",X1_power);
X1_plt.DataTipTemplate.DataTipRows(2) = dataTipTextRow("f",F);

subplot(2,1,2)
X2_plt = plot(F,10*log10(abs(X2(1:1024/2)).^2), "DisplayName", "Student 2" );
legend
ylim([-50,75])
% Both have the same frequency peak
% Selecting second highest for second signal
[X2_pkvals, X2_pklocs] =
findpeaks(X2_power,"MinPeakDistance",100,"MinPeakProminence",40);
X2_pkval = X2_pkvals(2);
X2_pkidx = X2_pklocs(2);
X2_pkfreq = F(X2_pkidx)

X2_pktip = datatip(X2_plt,'DataIndex',X2_pkidx,'Location','northwest');
X2_plt.DataTipTemplate.DataTipRows(1) = dataTipTextRow("P",X2_power);
X2_plt.DataTipTemplate.DataTipRows(2) = dataTipTextRow("f",F);

han=axes(fig,'visible','off');

```

```

han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Power (dB)');
xlabel(han,'Frequency (Hz)');
title(han,'Input Signal Frequency Response');
% Zooming in to peaks

fig = figure;

subplot(2,1,1)
X1_plt = plot(F,10*log10(abs(X1(1:1024/2)).^2), "DisplayName", "Student 1" );
legend
ylim([0,X1_pkval*1.1])
xlim([X1_pkfreq-250,X1_pkfreq+250])

subplot(2,1,2)
X2_plt = plot(F,10*log10(abs(X2(1:1024/2)).^2), "DisplayName", "Student 2" );
legend
ylim([-10,X2_pkval*1.1])
xlim([X2_pkfreq-250,X2_pkfreq+250])

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Power (dB)');
xlabel(han,'Frequency (Hz)');
title(han,'Frequency Response @ Relevant Peaks');
%%
%[text] ## Implementing Filter
%[text] "For each of the two signals, use Matlab filterDesigner to design a
bandstop FIR filter (stop band width of no more than 100Hz), with minimum
order and with quantisation of coefficients to 32-bit floating point numbers,
to remove the strongest component from the signal; the other components in the
signal should not be affected significantly, i.e. they should not be
distorted. If both signals have their strongest components in the same
frequency, then the second strongest component in the second signal should be
removed instead. In the report, provide the specifications of both filters,
along with appropriately labelled time domain and frequency domain (both
magnitude and phase) plots of the filters, and its pole-zero diagrams before
and after quantisation."
%filterDesigner

h1 = single(load("./X1_Band_Stop_Coeffs.mat").Num);
h2 = single(load("./X2_Band_Stop_Coeffs.mat").Num);
%%
%[text] ## Filter Response
fig = figure;
subplot(2,1,1)
plot(0:length(h1)-1,h1, "DisplayName", "Filter 1" );
xlim([0,length(h2)-1])

```



```

legend
subplot(2,1,2)
plot(0:length(h2)-1,h2, "DisplayName", "Filter 2" );
legend

xlim([0,length(h2)-1])
han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'h[n]');
xlabel(han,'n');
title(han,'Filter Coefficients');


fig = figure;
freqz(h1,1,F,fs);
fig = figure;
freqz(h2,1,F,fs);
fig = figure;
zplane(h1,1,"ctf")
title("Poles and Zeros of Filter 1")

fig = figure;
zplane(h2,1,"ctf")
title("Poles and Zeros of Filter 2")
fig = figure;
zplane(h2,1,"ctf")
title("Poles and Zeros of Filter 2 @ Unit Circle")
xlim([-1.5,1.5])
ylim([-1.5,1.5])
%%
%[text] ## Filtering The Signal
y1 = filter(h1,1,x1);
y2 = filter(h2,1,x2);


fig = figure;
subplot(2,1,1)
plot(t.*1e3,y1(1:length(t)), "DisplayName", "Student 1" );
legend

subplot(2,1,2)
plot(t.*1e3,y2(1:length(t)), "DisplayName", "Student 2" );
legend

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Amplitude');

```

```

xlabel(han,'Time (ms)');
title(han,'Filtered Signal Time Domain');

Y1 = fft(y1,1024);
Y1_power = 10*log10(abs(Y1(1:1024/2)).^2);

Y2 = fft(y2,1024);
Y2_power = 10*log10(abs(Y2(1:1024/2)).^2);

fig = figure;

subplot(2,1,1)
Y1_plt = plot(F,10*log10(abs(Y1(1:1024/2)).^2), "DisplayName", "Student 1" );
legend
ylim([-50,75])

subplot(2,1,2)
Y2_plt = plot(F,10*log10(abs(Y2(1:1024/2)).^2), "DisplayName", "Student 2" );
legend
ylim([-50,75])

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Power (dB)');
xlabel(han,'Frequency (Hz)');
title(han,'Filtered Signal Frequency Response');
% Zooming in to peaks

fig = figure;

subplot(2,1,1)
Y1_plt = plot(F,10*log10(abs(Y1(1:1024/2)).^2), "DisplayName", "Student 1" );
legend
xlim([X1_pkfreq-250,X1_pkfreq+250])

subplot(2,1,2)
y2_plt = plot(F,10*log10(abs(Y2(1:1024/2)).^2), "DisplayName", "Student 2" );
legend
xlim([X2_pkfreq-250,X2_pkfreq+250])

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Power (dB)');
xlabel(han,'Frequency (Hz)');
title(han,'Frequency Response @ Relevant Peaks');
%%

```

```

%[text] ## Exporting to C Header File
% From save_data.m

function save_filter(b_fir,a_fir,filename,prefix)
    fid = fopen(filename,'w');

    fprintf(fid,['#pragma once' char([13 10])]);
    fwrite(fid,char([13 10]),'uchar');

    %% FIR FILTER
    %% B
    fprintf(fid,['#define %s_N_FIR_B %d' char([13 10])],
prefix,length(b_fir));
    fwrite(fid,char([13 10]),'uchar');

    fwrite(fid,'float '+prefix+'_b_fir[] = { ', 'uchar');
    for ct = 1:length(b_fir)-1
        fprintf(fid,'%7ff, ', single(b_fir(ct)));
    end
    fprintf(fid,'%7ff, ', single(b_fir(end)));
    fwrite(fid,[' };' char([13 10])], 'uchar');
    fwrite(fid,char([13 10]), 'uchar');

    %% A
    fprintf(fid,['#define %s_N_FIR_A %d' char([13 10])],
prefix,length(a_fir));
    fwrite(fid,char([13 10]), 'uchar');
    fwrite(fid,'float '+prefix+'_a_fir[] = { ', 'uchar');
    for ct = 1:length(a_fir)-1
        fprintf(fid,'%7ff, ', single(a_fir(ct)));
    end
    fprintf(fid,'%7ff, ', single(a_fir(end)));
    fwrite(fid,[' };' char([13 10])], 'uchar');
    fwrite(fid,char([13 10]), 'uchar');

    fclose(fid);
end

save_filter(h1,1,"data_1.h","Filter_1")
save_filter(h2,1,"data_2.h","Filter_2")

h1_sfix = load("./X1_Band_Stop_Coeffs_sfix16.mat").Num;
h1_sfix = fi(h1_sfix,1,16,15);
h2_sfix = load("./X2_Band_Stop_Coeffs_sfix16.mat").Num;
h2_sfix = fi(h2_sfix,1,16,15);

x1_sfix = fi(x1/max(x1),1,16,15);
x2_sfix = fi(x2/max(x2),1,16,15);

y1_sfix = filter(h1_sfix,1,x1_sfix);
y2_sfix = filter(h2_sfix,1,x2_sfix);

```

```

mean( (y1 - double(y1_sfix.*max(x1))).^2 )
mean( (y2 - double(y2_sfix.*max(x2))).^2 )
% Now export as a fixed point integer

save_filter(h1_sfix,1,"data_1_fixed.h","Filter_1_Fixed")
save_filter(h2_sfix,1,"data_2_fixed.h","Filter_2_Fixed")

```

7.2 C Implementation

7.2.1 filter.h

```

/**
 * @file filter.h
 * @brief FIR filter structure definitions and functions.
 */

#ifndef FILTER_H
#define FILTER_H

#include <stdint.h>
#include <stdbool.h>

typedef float float32_t;
#define BUFF_SIZE          900U
#define BUFF_ZERO_SIZE     BUFF_SIZE + BUFF_SIZE / 2

typedef struct {
    uint8_t symmetric;
    uint32_t coeff_b_len;
    uint32_t coeff_a_len;
    float32_t *coeff_a_ptr;
    float32_t *coeff_b_ptr;
} filter_t;

/**
 * @brief Applies a FIR (Finite Impulse Response) filter to an input signal.
 *
 * @param[in] p_input Pointer to the input signal array
 * @param[in] p_input_len Length of the input signal
 * @param[in] p_filter Pointer to the FIR filter structure containing
coefficients
 * @param[out] p_output Pointer to the output signal array where filtered
results are stored
 *
 * @note The filter coefficients are accessed from p_filter->coeff_b_ptr with
length p_filter->coeff_b_len.
 * @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
 *
 * @return void
 */
void filter_signal(const float32_t *p_input, uint32_t p_input_len, const

```

```

filter_t *p_filter, float32_t *p_output);

/**
 * @brief Performs linear convolution of two integer arrays.
 * @note Output length will be (p_input1_len + p_input2_len - 1).
 *
 * @return void
 */
void convolution(const int32_t *p_input1, uint32_t p_input1_len, const int32_t
*p_input2, uint32_t p_input2_len, int32_t *p_output, uint32_t *p_output_len);

/**
 * @brief Applies a symmetric FIR filter to an input signal.
 *
 * This function implements a symmetric finite impulse response (FIR) filter,
 * taking advantage of coefficient symmetry to process the input signal.
 * For each output sample, the filter computes a weighted sum using symmetric
 * tap pairs.
 *
 * @param[in] p_input Pointer to the input signal array.
 * @param[in] p_input_len Length of the input signal.
 * @param[in] p_filter Pointer to the FIR filter structure containing
coefficients
 *
 * and filter length information.
 * @param[out] p_output Pointer to the output signal array where filtered
results
 *
 * are stored.
 *
 * @return void
 *
 * @note Filter coefficients must exhibit symmetry:  $h[k] = h[N-1-k]$ .
 * @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
 * @note Assumes odd length of filter coefficients
 */
void symm_filter_signal(const float32_t *p_input, uint32_t p_input_len, const
filter_t *p_filter, float32_t *p_output);

/**
 * @brief Converts an input signal to fixed-point and applies a symmetric FIR
filter to it.
 *
 * This function implements a scales and converts a floating-point input
signal to fixed-point integer.
 * Applies a symmetric FIR filter to it.
 * Converts the output signal back to floating-point.
 *
 * @param[in] p_input Pointer to the input signal array.
 * @param[in] p_input_len Length of the input signal.
 * @param[in] p_filter Pointer to the FIR filter structure containing
coefficients
 *
 * and filter length information.
 * @param[out] p_output Pointer to the output signal array where filtered

```

```

results
*
*                                     are stored.
*
* @return void
*
* @note Filter coefficients must exhibit symmetry:  $h[k] = h[N-1-k]$ .
* @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
* @note Assumes odd length of filter coefficients
*/
void symm_filter_signal_short(const float32_t *p_input, const uint32_t
p_input_len, const filter_t *p_filter, float32_t *p_output, const float32_t
p_coeff_norm);

/**
* @brief Applies a symmetric FIR filter to an integer input signal.
*
* This function implements a symmetric finite impulse response (FIR) filter,
* taking advantage of coefficient symmetry to process the input signal.
* For each output sample, the filter computes a weighted sum using symmetric
* tap pairs.
*
* @param[in] x Pointer to the input signal array.
* @param[in] p_input_len Length of the input signal.
* @param[in] N Number of filter coefficients
* @param[in] h Pointer to the filter coefficient array.
* @param[in] p_coeff_norm shift factor.
*
* @param[out] y Pointer to the output signal array where filtered results
*                                     are stored.
*
* @return void
*
* @note Filter coefficients must exhibit symmetry:  $h[k] = h[N-1-k]$ .
* @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
* @note Assumes odd length of filter coefficients
*/
void symm_filter_signal_short_nconv(const int16_t *x, const uint32_t
p_input_len, const uint32_t N, const int16_t *h, int32_t *y, const int16_t
p_coeff_norm);

/**
* @brief Converts an input signal to fixed-point and applies an FIR filter to
it using SIMD intrinsics.
*
* @param[in] p_input Pointer to the input signal array.
* @param[in] p_input_len Length of the input signal.
* @param[in] p_filter Pointer to the FIR filter structure containing
coefficients
*                                     and filter length information.
* @param[in] p_coeff_norm scaling and normalisation factor
* @param[out] p_output Pointer to the output signal array where filtered
results

```

```

*                                     are stored.
*
* @return void
*
* @note Filter coefficients must exhibit symmetry:  $h[k] = h[N-1-k]$ .
* @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
* @note Assumes odd length of filter coefficients
*/
void fir2_set_run(const float32_t *p_input, const uint32_t p_input_len, const
filter_t *p_filter, float32_t *p_output, const float32_t p_coeff_norm);

/**
 * @brief Applies an FIR filter to an integer input signal using SIMD
intrinsic.
 *
 * @param[in] x Pointer to the input signal array.
 * @param[in] h Pointer to the reversed filter coefficient array.
 * @param[out] y Pointer to the output signal array where filtered results
are stored.
 * @param[in] n Length of the input signal.
 * @param[in] m Number of filter coefficients
 * @param[in] s shift factor.
 *
 * @return void
 *
 * @note Filter coefficients must exhibit symmetry:  $h[k] = h[N-1-k]$ .
 * @note Filter coefficients p_filter->coeff_a are assumed = [ 1 ].
 * @note Assumes odd length of filter coefficients
 */
void fir2(const int x[], const int h[], short y[], int n, int m, int s);

#endif /* FILTER_H */

```

7.2.2 filter.c

```

#include "filter.h"
#include "util.h"
#include <stdio.h>

```

```

void filter_signal(const float32_t *p_input, uint32_t p_input_len, const
filter_t *p_filter, float32_t *p_output)
{
    const uint32_t N = p_filter->coeff_b_len;
    const float32_t *h = p_filter->coeff_b_ptr;

    // filter loop
    uint32_t n, k;
    for (n = 0; n < p_input_len; n++)
    {
        float32_t l_acc = 0.0f;

```

```

        // calculate tap-sums
        for (k = 0; k < N; k++) {
            if (n >= k) {
                l_acc += p_input[n - k] * h[k];
            }
            else break;
        }
        p_output[n] = l_acc;
    }
}

void convolution(const int32_t *p_input1, uint32_t p_input1_len, const int32_t
*p_input2, uint32_t p_input2_len, int32_t *p_output, uint32_t *p_output_len)
{
    *p_output_len = p_input1_len + p_input2_len - 1;

    // convolution loop
    uint32_t n, k;
    for (n = 0; n < *p_output_len; n++)
    {
        uint32_t l_acc = 0;

        // calculate tap-sums
        for (k = 0; k < p_input1_len; k++)
        {
            if (n >= k && (n - k) < p_input2_len)
            {
                l_acc += p_input1[k] * p_input2[n - k];
            }
        }
        p_output[n] = l_acc;
    }
}

void symm_filter_signal(const float32_t *p_input, uint32_t p_input_len, const
filter_t *p_filter, float32_t *p_output)
{
    const uint32_t N = p_filter->coeff_b_len;
    const float32_t *h = p_filter->coeff_b_ptr;

    const uint32_t l_half_len = (N / 2);

    // filter loop
    uint32_t n, k;
    for (n = 0; n < p_input_len; n++)
    {
        float32_t l_acc = 0.0f;

        // loop through the first half of coefficients
        for (k = 0; k < l_half_len; k++) {

```



```

        // add symmetric taps before multiplying
        float32_t tap_sum = 0.0f;
        if (n >= k)
        {
            tap_sum += p_input[n - k];
        }
        if (n >= (N - 1 - k))
        {
            tap_sum += p_input[n - (N - 1 - k)];
        }
        // calculate tap-sums
        l_acc += tap_sum * h[k];
    }

    // handle the middle tap if N is odd (assumed filter has odd length)
    if (n >= l_half_len)
    {
        l_acc += p_input[n - l_half_len] * h[l_half_len];
    }
    p_output[n] = l_acc;
}
}

```

```

static void symm_filter_signal_short_nconv(const int16_t *x, const uint32_t
p_input_len, const uint32_t N, const int16_t *h, int32_t *y, const int16_t
p_coeff_norm)
{
    const uint32_t l_half_len = (N / 2);

    // filter loop
    uint32_t n, k;
    for (n = 0; n < p_input_len; n++)
    {
        int32_t l_acc = 0;

        // loop through the first half of coefficients
        for (k = 0; k < l_half_len; k++) {

            // add symmetric taps before multiplying
            int32_t tap_sum = 0;
            if (n >= k)
            {
                tap_sum += x[n - k];
            }
            if (n >= (N - 1 - k))
            {
                tap_sum += x[n - (N - 1 - k)];
            }
            // calculate tap-sums
            l_acc += tap_sum * h[k];
        }
    }
}

```

```

    }

    // handle the middle tap if N is odd (assumed filter has odd length)
    if (n >= l_half_len) {
        l_acc += (int32_t)x[n - l_half_len] * h[l_half_len];
    }

    // rounding
    // add half-LSB (1 << (s - 1)) to round to nearest integer during
shift
    l_acc += (1 << (p_coeff_norm - 1));

    // re-normalization
    l_acc >>= p_coeff_norm;

    y[n] = l_acc;
}
}

```

```

void symm_filter_signal_short(const float32_t *p_input, const uint32_t
p_input_len, const filter_t *p_filter, float32_t *p_output, const float32_t
p_coeff_norm)
{
    const uint32_t N = p_filter->coeff_b_len;

    // convert coefficients to int
    static int16_t h_q[BUFF_SIZE] = { 0 };
    float_to_int16_normalized(p_filter->coeff_b_ptr, N, h_q, p_coeff_norm);

    // convert input to int
    static int16_t x_q[BUFF_SIZE] = { 0 };
    float_to_int16_normalized(p_input, p_input_len, x_q, p_coeff_norm);

    static int32_t y_q[BUFF_SIZE] = { 0 };
    // filter loop
    symm_filter_signal_short_nconv(x_q, p_input_len, N, h_q, y_q, 1);

    // convert output to float
    int32_to_float(y_q, p_input_len, p_output, 1, p_coeff_norm);
}

```

```

void fir2_set_run(const float32_t *p_input, const uint32_t p_input_len, const
filter_t *p_filter, float32_t *p_output, const float32_t p_coeff_norm)
{
    const uint32_t N = p_filter->coeff_b_len;
    const uint32_t M = p_input_len;
    const uint32_t L_even = (N % 2U == 0U) ? N : (N + 1U);
    const uint32_t M_even = (M % 2U == 0U) ? M : (M + 1U);

```

```

// right-shift used by fir2() before storing to int16.
//      Q15xQ15 -> Q30 accumulation -> Q15 output.
const int s = 15;

// convert coefficients
static int16_t h_q[BUFF_SIZE] = { 0 };
float_to_int16_normalized(p_filter->coeff_b_ptr, N, h_q, p_coeff_norm);

// convert input
static int16_t x_q[BUFF_SIZE] = { 0 };
float_to_int16_normalized(p_input, M, x_q, p_coeff_norm);

// build padded input of length (L_even + M_even) samples
const uint32_t x_pad_len = L_even + M_even ;
static int16_t x_pad[BUFF_ZERO_SIZE] = { 0 };

uint32_t i;
for(i = 0; i < (L_even - 1U); i++)
{
    x_pad[i] = 0;
}
for(i = 0; i < M; i++)
{
    x_pad[(L_even - 1U) + i] = x_q[i];
}
for(i = (L_even - 1U) + M; i < x_pad_len; i++)
{
    x_pad[i] = 0;
}

// build padded and reversed coefficient array
static int16_t h_pad[BUFF_ZERO_SIZE] = { 0 };
for(i = 0; i < N; i++)
{
    h_pad[(L_even - 1U) - i] = h_q[i];
}
for(i = N; i < L_even; i++)
{
    h_pad[(L_even - 1U) - i] = 0;
}

static int16_t y_q[BUFF_ZERO_SIZE + 300] = { 0 };

// ensure 4-byte word alignment
_nassert((((int)x_pad) & 0x3) == 0);
_nassert((((int)h_pad) & 0x3) == 0);
_nassert((((int)y_q) & 0x3) == 0);

// filter loop
fir2((const int *)x_pad, (const int *)h_pad, y_q, (int)L_even,
(int)M_even, s);

```

```

    // x and h were both scaled by p_coeff_norm => scaled by p_coeff_norm^2.
    // fir2 stores (acc >> s) => reconstruct by multiplying by 2^s.
    const float32_t inv_scale = (float32_t)(1U << s) / (p_coeff_norm *
p_coeff_norm);

    for(i = 0; i < M; i++)
    {
        p_output[i] = (float32_t)y_q[i] * inv_scale;
    }
}

// extracted from Example 3-14. FIR Filter- Optimized Form in C6x programmer's
guide(198d)
void fir2(const int x[], const int h[], short y[], int n, int m, int s)
{
    int i, j;
    long y0, y1;
    long round = 1L << (s - 1);

    #pragma MUST_ITERATE (,,2);
    for (j = 0; j < (m >> 1); j++)
    {
        y0 = y1 = round;
        #pragma MUST_ITERATE (,,2);
        for (i = 0; i < (n >> 1); i++)
        {
            y0 += _mpy (x[i + j], h[i]);
            y0 += _mpyh (x[i + j], h[i]);
            y1 += _mpyh1(x[i + j], h[i]);
            y1 += _mpylh(x[i + j + 1], h[i]);
        }
        *y++ = (int)(y0 >> s);
        *y++ = (int)(y1 >> s);
    }
}

```

7.2.3 util.h

```

/**
 * @file util.h
 * @brief utility functions for FIR filter implementations.
 */

#ifndef UTIL_H
#define UTIL_H

#include <stdint.h>
#include <time.h>

typedef float float32_t;

// profiling definitions

```

```

#undef CLOCKS_PER_SEC
#define CLOCKS_PER_SEC (300000000)
extern cregister volatile unsigned int TSCL;
extern cregister volatile unsigned int TSCH;

/**
 * @brief Returns current clock cycle.
 *
 * @return clock_t
 */
clock_t clock(void);

/**
 * @brief Calculates the mean (average) of an array of unsigned integers.
 *
 * @param[in] p_input Pointer to an array of uint32_t values to be averaged.
 * @param[in] p_input_len The number of elements in the input array.
 * @param[out] p_output Pointer to a float32_t variable where the calculated
mean will be stored.
 *
 * @return void
 */
void calculate_mean(const uint32_t *p_input, uint32_t p_input_len, float32_t
*p_output);

/**
 * @brief Generates a signal by repeating and mean-centering an input array.
 *
 * @param p_input Pointer to the input array of uint32_t values.
 * @param p_input_len Length of the input array.
 * @param p_output Pointer to the output array of float32_t values where
the generated signal will be stored.
 * @param p_output_len Length of the output array. Should be a multiple of
p_input_len.
 *
 * @return void
 */
void generate_signal(const uint32_t *p_input, uint32_t p_input_len, float32_t
*p_output, uint32_t p_output_len);

/**
 * @brief Normalises an array of floats to an array of signed 16-bit fixed-
point integers.
 * @param input: The IEEE 754 single precision float.
 * @param scale: The scaling factor (e.g., 32767.0f for full range).
 * @return: A saturated 16-bit signed integer.
 */

```

```

void float_to_int16_normalized(const float32_t *p_input, const uint32_t
p_input_len, int16_t *p_output, const float32_t p_scale);

/**
 * @brief Converts an array of scaled 16-bit signed integers back to an array
of 32-bit floats.
 * @param input The int16_t value to convert.
 * @param scale The factor the float was multiplied by (e.g., 100.0f).
 * @return float The reconstructed float value.
 */
void int16_to_float(const int16_t *p_input, const uint32_t p_input_len,
float32_t *p_output, const int16_t s, const float32_t p_scale);

/**
 * @brief Converts an array of scaled 32-bit signed integers back to an array
of 32-bit floats.
 * @param input The int32_t value to convert.
 * @param scale The factor the float was multiplied by (e.g., 100.0f).
 * @return float The reconstructed float value.
 */
void int32_to_float(const int32_t *p_input, const uint32_t p_input_len,
float32_t *p_output, const int16_t s, const float32_t p_scale);

/**
 * @brief Prints a formatted table of two float arrays with their differences
and statistical summary.
 *
 * @param p_output1 Pointer to the first float32_t array
 * @param p_output2 Pointer to the second float32_t array
 * @param p_output_len Total length of both arrays
 * @param p_print_start Starting index for the table output (inclusive)
 * @param p_print_n Number of elements to print in the table
 *
 * @details
 * This function displays:
 * - A formatted table with columns for Index, y1, y2, and their absolute
difference
 *   for elements from p_print_start to min(p_print_start + p_print_n,
p_output_len)
 * - A statistics summary including:
 *   - Mean value of y1 (calculated over entire array)
 *   - Mean value of y2 (calculated over entire array)
 *   - Mean absolute difference between arrays
 *   - Total absolute difference across all elements
 */
void print_statistics(const float32_t *p_output1, const float32_t *p_output2,
uint32_t p_output_len, uint32_t p_print_start, uint32_t p_print_end);

/**
 * @brief Prints a set of array samples from index 0
 *
 * @param x Pointer to the array of float32_t values to be written.

```

```

    * @param n The number of elements to print.
    *
    * @return void
    *
    */
void print_results(const float32_t *x, const uint32_t n);

/**
 * @brief Records an array of floating-point values to a CSV file.
 *
 * @param p_output Pointer to the array of float32_t values to be written.
 * @param p_output_len The number of elements in the p_output array.
 * @param p_filename The path to the output file. The file will be created or
overwritten.
 *
 * @return void
 *
 * @note If the file cannot be opened, an error message is printed to stdout
and the
 *       function returns without writing any data.
 */
void record_output(const float32_t *p_output, uint32_t p_output_len, const
char *p_filename);

#endif /* UTIL_H */

```

7.2.4 util.c

```

#include "util.h"
#include <stdio.h>
#include <math.h>
#include <string.h>

clock_t clock(void)
{
    unsigned int low = TSCL;
    unsigned int high = TSCH;
    if(high) return (clock_t)-1;
    return low;
}

void calculate_mean(const uint32_t *p_input, uint32_t p_input_len, float32_t
*p_output)
{
    uint32_t l_sum = 0;

    uint32_t i;
    for(i = 0; i < p_input_len; i++)
    {
        l_sum += p_input[i];
    }

    *p_output = (float32_t) l_sum / p_input_len;
}

```

```
}
```

```
void generate_signal(const uint32_t *p_input, uint32_t p_input_len, float32_t  
*p_output, uint32_t p_output_len)
```

```
{
```

```
    float32_t l_mean = 0;
```

```
    calculate_mean(p_input, p_input_len, &l_mean);
```

```
    // generate the first block
```

```
    uint32_t i;
```

```
    for(i = 0; i < p_input_len; i++)
```

```
    {
```

```
        p_output[i] = (float32_t) p_input[i] - l_mean;
```

```
    }
```

```
    const uint32_t l_repetitions = p_output_len / p_input_len;
```

```
    const size_t l_block_size = p_input_len * sizeof(float32_t);
```

```
    // copy the first block across the rest of the array
```

```
    for (i = 1; i < l_repetitions; i++)
```

```
    {
```

```
        // Copy the first block into the current repetition's offset
```

```
        memcpy(&p_output[i * p_input_len], p_output, l_block_size);
```

```
    }
```

```
}
```

```
void float_to_int16_normalized(const float32_t *p_input, const uint32_t  
p_input_len, int16_t *p_output, const float32_t p_scale)
```

```
{
```

```
    uint16_t i;
```

```
    for(i = 0; i < p_input_len; i++)
```

```
    {
```

```
        // scale sample
```

```
        float32_t scaled_val = p_input[i] * p_scale;
```

```
        // round to nearest integer
```

```
        scaled_val = roundf(scaled_val);
```

```
        // saturate to prevent overflow
```

```
        if (scaled_val >= 32767.0f) {
```

```
            p_output[i] = 32767;
```

```
            continue;
```

```
        } else if (scaled_val <= -32768.0f) {
```

```
            p_output[i] = -32768;
```

```
            continue;
```

```
        }
```

```
        p_output[i] = (int16_t)scaled_val;
```

```
    }
```

```
}
```



```

void int16_to_float(const int16_t *p_input, const uint32_t p_input_len,
float32_t *p_output, const int16_t s, const float32_t p_scale)
{
    const float32_t l_inv_scale = (float32_t)(1U << s) / (p_scale * p_scale);

    uint16_t i;
    for(i = 0; i < p_input_len; i++)
    {
        p_output[i] = (float32_t)p_input[i] * l_inv_scale;
    }
}

```

```

void int32_to_float(const int32_t *p_input, const uint32_t p_input_len,
float32_t *p_output, const int16_t s, const float32_t p_scale)
{
    const float32_t l_inv_scale = (float32_t)(1U << s) / (p_scale * p_scale);

    uint16_t i;
    for(i = 0; i < p_input_len; i++)
    {
        p_output[i] = (float32_t)p_input[i] * l_inv_scale;
    }
}

```

```

void print_statistics(const float32_t *p_output1, const float32_t *p_output2,
uint32_t p_output_len, uint32_t p_print_start, uint32_t p_print_n)
{
    const uint32_t p_print_end = (p_print_start + p_print_n < p_output_len) ?
(p_print_start + p_print_n) : p_output_len;

    printf("| Index |      y1      |      y2      | Difference |\n");
    printf("|-----|-----|-----|-----|\n");
    uint32_t i;
    for(i = p_print_start; i < p_print_end; i++)
    {
        printf("| %3d | %12.6f | %12.6f | %12.6f |\n", i, p_output1[i],
p_output2[i], fabsf(p_output1[i] - p_output2[i]));
    }

    // Statistics summary
    float32_t mean_y1 = 0, mean_y2 = 0, mean_diff = 0, total_diff = 0;
    for(i = 0; i < p_output_len; i++)
    {
        mean_y1 += p_output1[i];
        mean_y2 += p_output2[i];
        total_diff += fabsf(p_output1[i] - p_output2[i]);
    }
    mean_y1 /= p_output_len;

```

```

    mean_y2 /= p_output_len;
    mean_diff = total_diff / p_output_len;
    printf("Statistics Summary:\n");
    printf("Mean of y1: %f\n", mean_y1);
    printf("Mean of y2: %f\n", mean_y2);
    printf("Mean of Differences: %f\n", mean_diff);
    printf("Total of Differences: %f\n", total_diff);
}

void print_results(const float32_t *x, const uint32_t n)
{
    printf("\nresult");
    uint8_t i;
    for(i = 0; i<10; i++)
    {
        printf("[%d] = %f ", i, x[i]);
    }
}

void record_output(const float32_t *p_output, uint32_t p_output_len, const
char *p_filename)
{
    FILE *l_file = fopen(p_filename, "w");

    if (l_file == NULL)
    {
        printf("Error. Not able to open file %s for writing.\n", p_filename);
        return;
    }

    uint32_t i;
    for (i = 0; i < p_output_len; i++)
    {
        fprintf(l_file, "%f,", p_output[i]);
    }

    // remove trailing comma (functions not supported on C67x device)
    // fseek(l_file, -1, SEEK_END);
    // ftruncate(fileno(l_file), ftell(l_file));
    printf("\nData saved in file: %s", p_filename);

    fclose(l_file);
}

```

7.2.5 main.c

```

#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include "util.h"
#include "filter.h"

```

```

#include "data.h"
#include "data_1_fixed.h"
#include "data_2_fixed.h"

/* DEFINES */
// file paths for data outputs
#define DATA_DIR "/home/preben/Documents/ee580_dsp/EE580-Filtering-Assignment/C_dsp/data/"
#define DATA_FILE_1 DATA_DIR "data1.txt"
#define DATA_FILE_2 DATA_DIR "data2.txt"

// signal generation declarations
#define REG_LENGTH 9U
#define REG1_LAST4 8874U
#define REG2_LAST4 4642U

// convolution buffer
#define SAMPLE_LEN 4

/* GLOBAL DECLARATIONS */
static uint32_t g_reg1[REG_LENGTH] = { 2, 0, 2, 1, 1, 8, 8, 7, 4 }; // 202118874
static uint32_t g_reg2[REG_LENGTH] = { 2, 0, 2, 1, 1, 4, 6, 4, 2 }; // 202114642

// buffers for data input and output
#pragma DATA_ALIGN(g_x, 8); // Align to 8-byte (64-bit) boundary
static float32_t g_x[BUFF_ZERO_SIZE] = { 0 }; // stack overflow if not static
#pragma DATA_ALIGN(g_y, 8); // Align to 8-byte (64-bit) boundary
static float32_t g_y[BUFF_ZERO_SIZE] = { 0 };

/* STATIC DECLARATIONS */
static void profile_filter_implementations(const float32_t *p_input, const
uint32_t p_input_len, const filter_t *p_filter, float32_t *p_output);

static double ticks_to_seconds(clock_t p_start, clock_t p_end)
{
    return (double) (p_end - p_start) / CLOCKS_PER_SEC;
}

/**
 * main.c
 * - declare FIR filter structs
 * - filter signal x1
 * - filter signal x2
 * - profile various FIR filter implementations
 */
int main(void)

```

```

{
    TSCL = 0;    // this starts the clock

    filter_t l_FIR_1;
    l_FIR_1.symmetric = true;
    l_FIR_1.coeff_b_len = Filter_1_Fixed_N_FIR_B;
    l_FIR_1.coeff_a_len = Filter_1_Fixed_N_FIR_A;
    l_FIR_1.coeff_a_ptr = Filter_1_Fixed_a_fir;
    l_FIR_1.coeff_b_ptr = Filter_1_Fixed_b_fir;

    filter_t l_FIR_2;
    l_FIR_2.symmetric = true;
    l_FIR_2.coeff_b_len = Filter_2_Fixed_N_FIR_B;
    l_FIR_2.coeff_a_len = Filter_2_Fixed_N_FIR_A;
    l_FIR_2.coeff_a_ptr = Filter_2_Fixed_a_fir;
    l_FIR_2.coeff_b_ptr = Filter_2_Fixed_b_fir;

    // filter signal x1
    generate_signal(g_reg1, REG_LENGTH, g_x, BUFF_SIZE);
    filter_signal(g_x, BUFF_SIZE, &l_FIR_1, g_y);
    record_output(g_y, BUFF_SIZE, DATA_FILE_1);

    // filter signal x2
    generate_signal(g_reg2, REG_LENGTH, g_x, BUFF_SIZE);
    filter_signal(g_x, BUFF_SIZE, &l_FIR_2, g_y);
    record_output(g_y, BUFF_SIZE, DATA_FILE_2);

    // profile FIR implementations
    profile_filter_implementations(g_x, BUFF_SIZE, &l_FIR_1, g_y);

    // convolution of 4 integer samples signals
    int32_t l_sample_signal1[SAMPLE_LEN] = { 1, 1, 1, 1 };
    int32_t l_sample_signal2[SAMPLE_LEN] = { 1, 2, 1, 2 };
    int32_t l_conv_output[SAMPLE_LEN * 2] = { 0 };
    uint32_t l_conv_len = 0;

    convolution(l_sample_signal1, SAMPLE_LEN, l_sample_signal2, SAMPLE_LEN,
l_conv_output, &l_conv_len);

    return 0;
}

static void profile_filter_implementations(const float32_t *p_input, const
uint32_t p_input_len, const filter_t *p_filter, float32_t *p_output)
{
    // CPU cycle profiling
    clock_t start, stop, overhead;
    start = clock();
    stop = clock();
    overhead = stop - start;

```

```

// number of output samples to print
const uint32_t l_samples_to_print = 10;

// calculation of scaling norm for fixed-point integer conversions
float32_t l_max = 0;
uint16_t i;
// find max sample value
for(i = 0; i < p_input_len; i++)
{
    float32_t l_sample = abs(p_input[i]);
    if(l_sample > l_max) l_max = l_sample;
}
// find max coefficient value
for(i = 0; i < p_filter->coeff_b_len; i++)
{
    float32_t l_sample = abs(p_filter->coeff_b_ptr[i]);
    if(l_sample > l_max) l_max = l_sample;
}
const float32_t l_scaled_norm = 32767.0f / (l_max);
printf("\nscaled norm = %f \n", l_scaled_norm);

// standard filter
start = clock();
filter_signal(p_input, p_input_len, p_filter, p_output);
stop = clock();
printf("\nfilter cycles: %d, seconds: %f", stop - start - overhead,
ticks_to_seconds(start, stop - overhead ));
record_output(p_output, p_input_len, DATA_DIR "data_standard.txt");
print_results(p_output, l_samples_to_print);

// symmetric filter
start = clock();
symm_filter_signal(p_input, p_input_len, p_filter, p_output);
stop = clock();
printf("\nsymm cycles: %d, seconds: %f", stop - start - overhead,
ticks_to_seconds(start, stop - overhead ));
record_output(p_output, p_input_len, DATA_DIR "data_symmetric.txt");
print_results(p_output, l_samples_to_print);

// symmetric filter with fixed-point
start = clock();
symm_filter_signal_short(p_input, p_input_len, p_filter, p_output,
l_scaled_norm);
stop = clock();
printf("\nsymm short cycles: %d, seconds: %f", stop - start - overhead,
ticks_to_seconds(start, stop - overhead ));
record_output(p_output, p_input_len, DATA_DIR "data_symmetric_fixed.txt");
print_results(p_output, l_samples_to_print);

// SIMD filter with fixed-point
start = clock();
fir2_set_run(p_input, p_input_len, p_filter, p_output, l_scaled_norm);

```

```

        stop = clock();
        printf("\nfir2 cycles: %d, seconds: %f", stop - start - overhead,
ticks_to_seconds(start, stop - overhead ));
        record_output(p_output, p_input_len, DATA_DIR "data_fixed_simd.txt");
        print_results(p_output, l_samples_to_print);
    }

```

7.3 MATLAB Comparison

```

%[text] ## Comparison of C Filtering
%TODO: Change path to board output
path = "/home/preben/Documents/ee580_dsp/EE580-Filtering-Assignment/C_dsp/
data/"
y1_c = readmatrix(path + "data1.txt");
y2_c = readmatrix(path + "data2.txt");

%%
%[text] ### Time Domain
%[text] As before we shall plot the first 3 cycles.
fig = figure;
subplot(2,1,1)
plot(t.*1e3,y1_c(1:length(t)), "DisplayName", "Student 1" );
legend

subplot(2,1,2)
plot(t.*1e3,y2_c(1:length(t)), "DisplayName", "Student 2" );
legend
han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Amplitude');
xlabel(han,'Time (ms)');
title(han,'Filtered Signal Time Domain (C)');
saveas(fig, 'filtered_time_c.svg');
%[text] Now let's plot the error between the matlab and C implementations
y1_dsp = readmatrix(path + "data_standard.txt");
y2_dsp = readmatrix(path + "data_symmetric.txt");
y3_dsp = readmatrix(path + "data_symmetric_fixed.txt");
y4_dsp = readmatrix(path + "data_fixed_simd.txt");

figure
subplot(5,1,1)
plot(1:length(y1),y1, "DisplayName", "y1" );
legend
subplot(5,1,2)
plot(1:length(y1),y1_dsp, "DisplayName", "y1_dsp" );
legend
subplot(5,1,3)
plot(1:length(y1),y2_dsp, "DisplayName", "y2_dsp" );
legend
subplot(5,1,4)

```

```

plot(1:length(y1),y3_dsp, "DisplayName", "y3_dsp" );
legend
subplot(5,1,5)
plot(1:length(y1),y4_dsp, "DisplayName", "y4_dsp" );
legend
hold off

y1_dsp_error = y1(1:length(y1)) - y1_dsp(1:length(y1));
y2_dsp_error = y1(1:length(y1)) - y2_dsp(1:length(y1));
y3_dsp_error = y1(1:length(y1)) - y3_dsp(1:length(y1));
y4_dsp_error = y1(1:length(y1)) - y4_dsp(1:length(y1));

fprintf("MSE of standard = %.3e", mean(y1_dsp_error.^2))
fprintf("MSE of symmetric = %.3e", mean(y2_dsp_error.^2))
fprintf("MSE of symmetric fixed = %.3e", mean(y3_dsp_error.^2))
fprintf("MSE of fixed simd = %.3e", mean(y4_dsp_error.^2))

y1_c_error = y1(1:length(t)) - y1_c(1:length(t));
y2_c_error = y2(1:length(t)) - y2_c(1:length(t));

fig = figure;
subplot(2,1,1)
plot(t.*1e3,y1_c_error, "DisplayName", "Student 1" );
legend

subplot(2,1,2)
plot(t.*1e3,y2_c_error, "DisplayName", "Student 2" );
legend
han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'MATLAB Amplitude - C Amplitude');
xlabel(han,'Time (ms)');
title(han,'Filtered Signal Time Domain Error');
saveas(fig, 'filtered_time_error.svg');

y1_c_error = y1(1:length(y1)) - y1_c(1:length(y1));
y2_c_error = y2(1:length(y2)) - y2_c(1:length(y2));

fprintf("MSE of y1 = %.3e", mean(y1_c_error.^2))
fprintf("MSE of y2 = %.3e", mean(y2_c_error.^2))
%%
%[text] ### Frequency Domain
%[text] 1024 point FFT
Y1_c = fft(y1_c,1024);
Y1_c_power = 10*log10(abs(Y1_c(1:1024/2)).^2);

Y2_c = fft(y2_c,1024);
Y2_c_power = 10*log10(abs(Y2_c(1:1024/2)).^2);

fig = figure;

```

```

subplot(2,1,1)
Y1_c_plt = plot(F,10*log10(abs(Y1_c(1:1024/2)).^2), "DisplayName", "Student
1" );
legend
ylim([-50,75])

subplot(2,1,2)
Y2_c_plt = plot(F,10*log10(abs(Y2_c(1:1024/2)).^2), "DisplayName", "Student
2" );
legend
ylim([-50,75])
% Both have the same frequency peak
% Selecting second highest for second signal

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'Power (dB)');
xlabel(han,'Frequency (Hz)');
title(han,'Filtered Signal Frequency Response (C)');
saveas(fig, 'filtered_freq_c.svg');

%[text] Frequency Error
Y1_c_error = Y1 - Y1_c;
Y2_c_error = Y2 - Y2_c;

fig = figure;

subplot(2,1,1)
Y1_c_plt = plot(F,Y1_c_error(1:512), "DisplayName", "Student 1" );
legend

subplot(2,1,2)
Y2_c_plt = plot(F,Y2_c_error(1:512), "DisplayName", "Student 2" );
legend
% Both have the same frequency peak
% Selecting second highest for second signal

han=axes(fig,'visible','off');
han.Title.Visible='on';
han.XLabel.Visible='on';
han.YLabel.Visible='on';
ylabel(han,'MATLAB Amplitude - C Amplitude');
xlabel(han,'Frequency (Hz)');
title(han,'Filtered Frequency Response Error');
saveas(fig, 'filtered_freq_error.svg');

```



```
fprintf("MSE of Y1 = %.3e", mean(Y1_c_error.^2))  
fprintf("MSE of Y2 = %.3e", mean(Y2_c_error.^2))
```

8 Bibliography

9 Contributions

Student, Daniel Stebbings:

- Responsible for the filter design and MATLAB code.
- Contribution: 50%

Student, Preben Rasamanickam:

- Responsible for the C code.
- Contribution: 50%